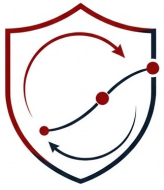
Principles of Robot Autonomy I

Planning and Control: Trajectory optimization, direct and shooting methods, LQR

Stanford University



SIA Lab
Safe and Intelligent Autonomy Lab

IPRL

Principles of Robot Autonomy I

Announcements:

- Lab section 2 this week
- HW 1 due this Thurs, Oct 8
- Slides uploaded to canvas before lecture
- Total enrolled: 160, total waitlist: 50

Lecture 5:

- PRM*, RRT* and re-wiring
- Trajectory optimization
- Direct, indirect, and shooting methods
- LQR, iLQR

Stanford University

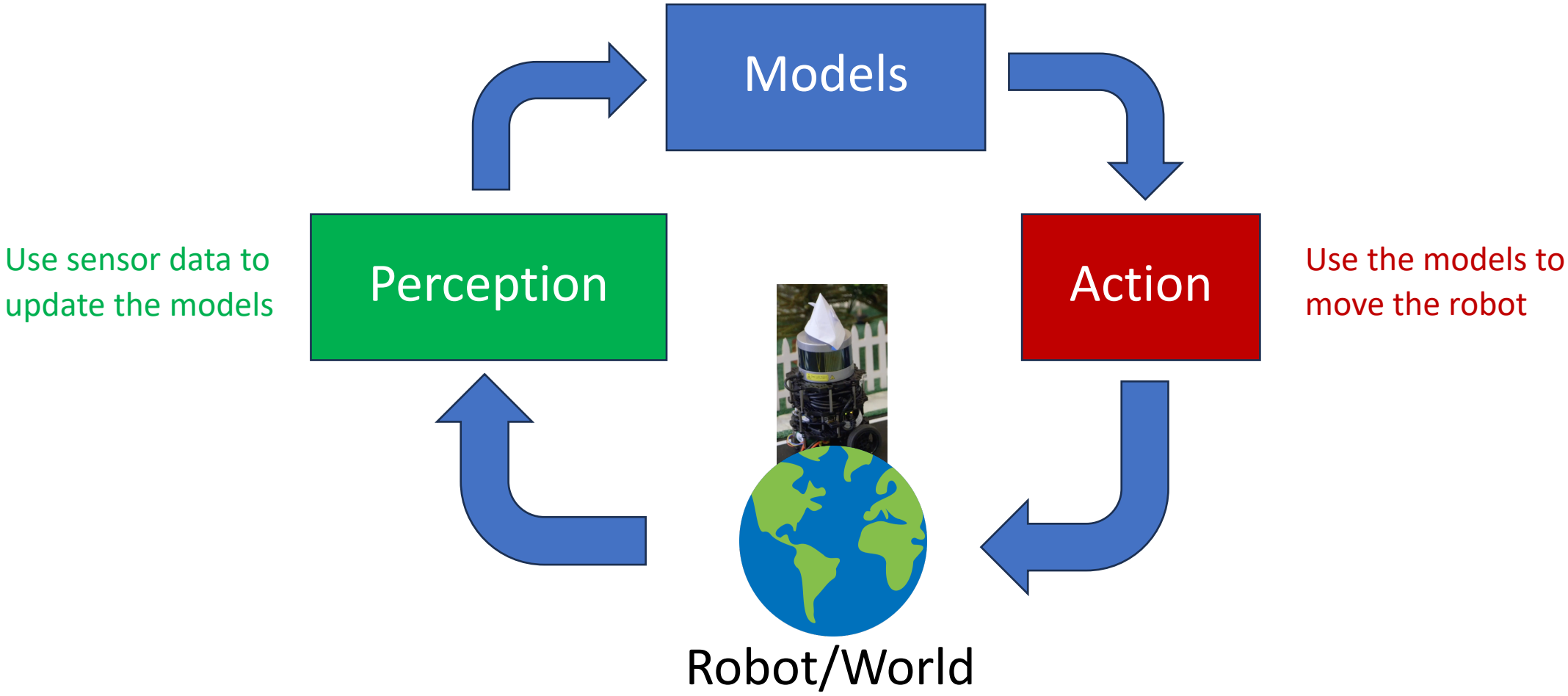
SIA Lab
Safe and Intelligent Autonomy Lab

MSL

IPRL

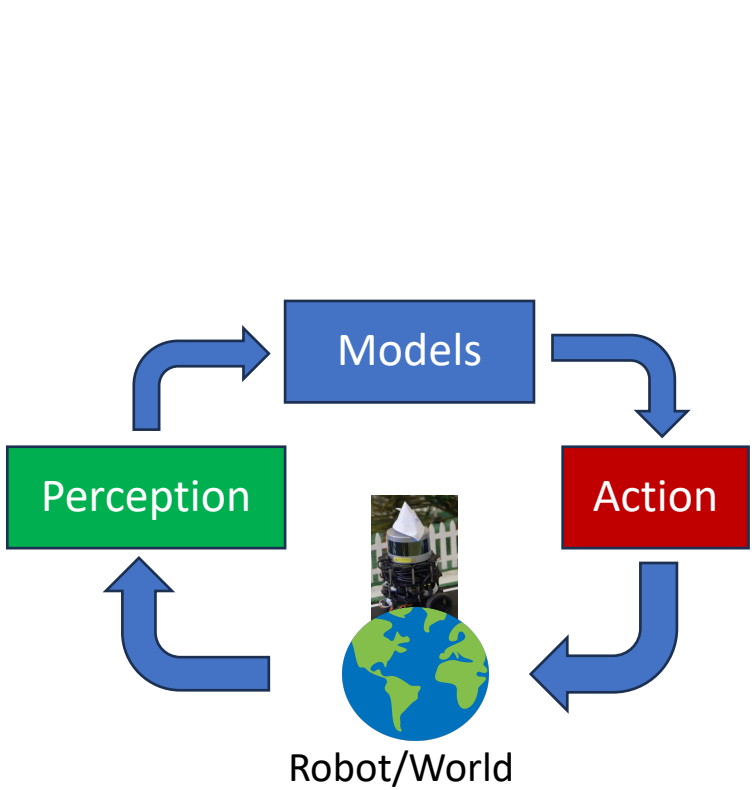
ASL
Autonomous Systems Lab

Full Stack Autonomy: the Perception-Action Loop



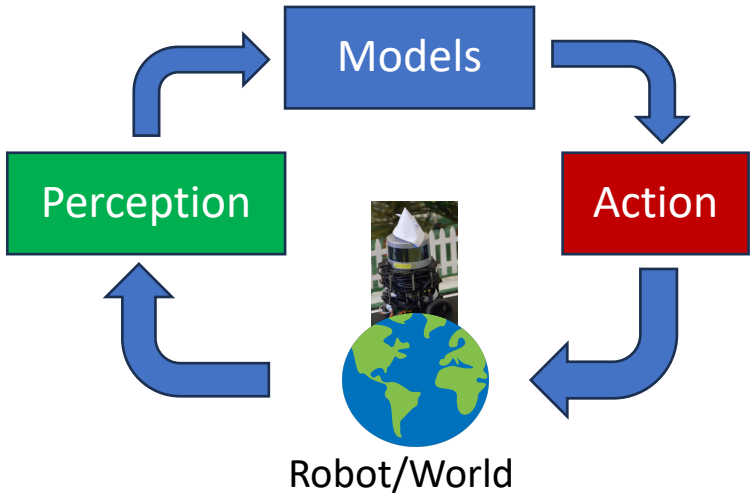
Action: Planning and Control Stack

Use the models to move the robot



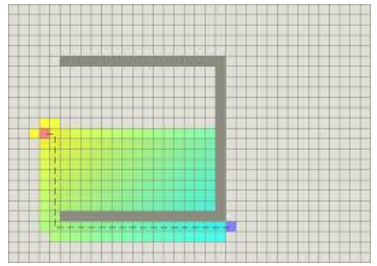
Action: Planning and Control Stack

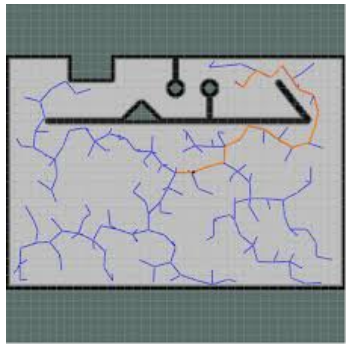
Use the models to move the robot



Path planning:

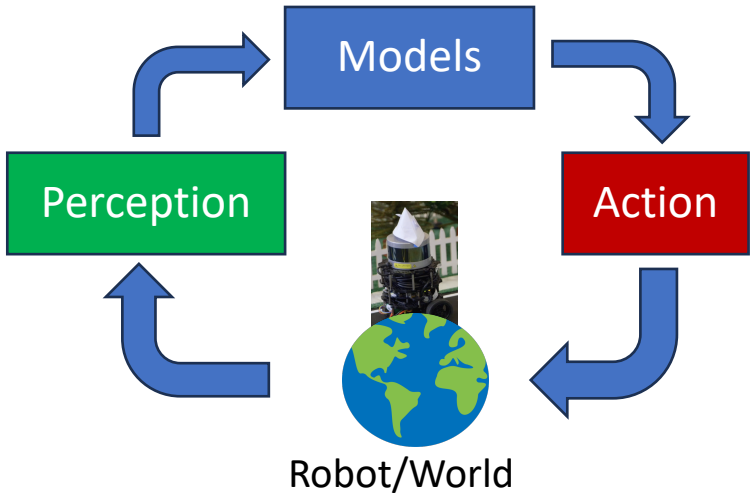
- A* planning on a graph
- RRT, RRT*





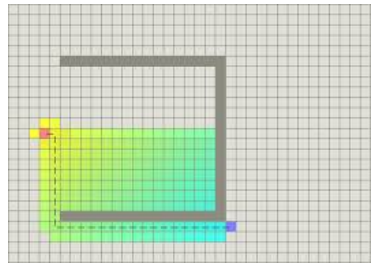
Action: Planning and Control Stack

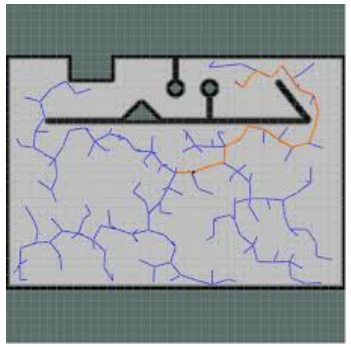
Use the models to move the robot



Path planning:

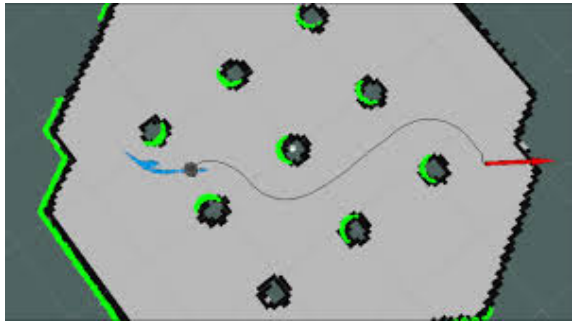
- A* planning on a graph
- RRT, RRT*





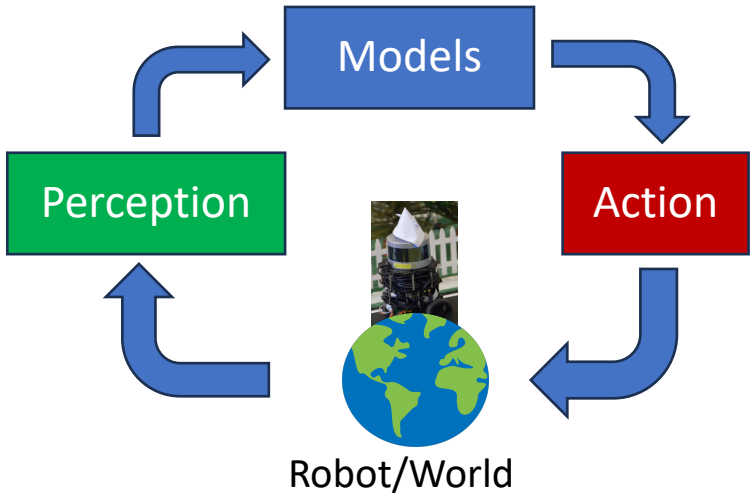
Trajectory optimization:

- Optimal control
- Numerical solvers



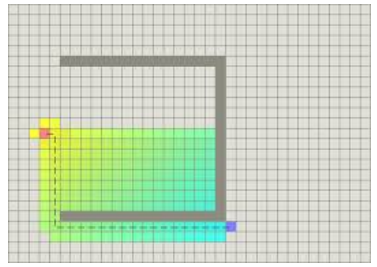
Action: Planning and Control Stack

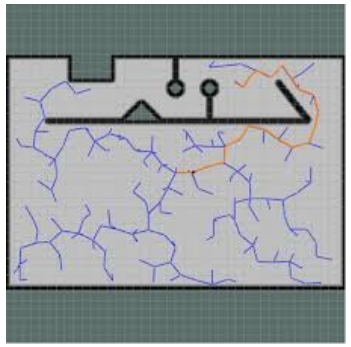
Use the models to move the robot



Path planning:

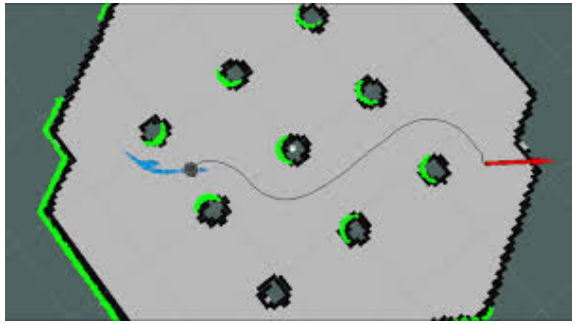
- A* planning on a graph
- RRT, RRT*





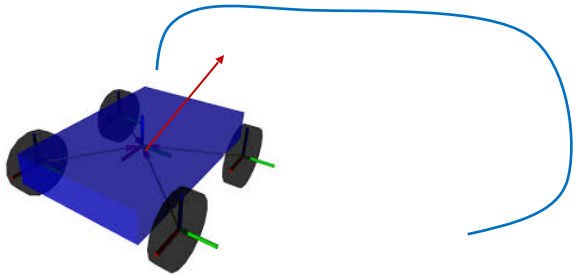
Trajectory optimization:

- Optimal control
- Numerical solvers



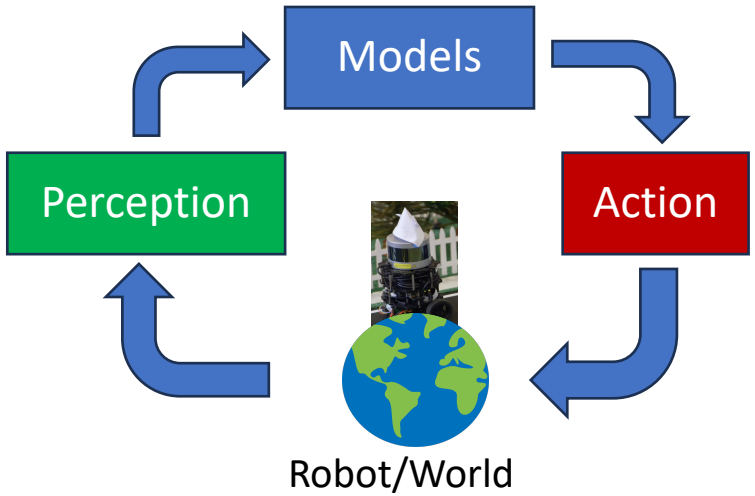
Trajectory following:

- Kinematic/dynamic models
- PID control
- LQR, gain scheduled LQR



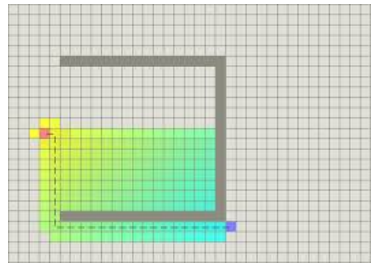
Action: Planning and Control Stack

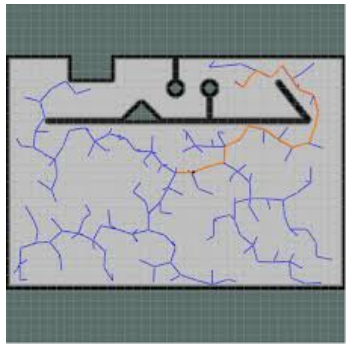
Use the models to move the robot



Path planning:

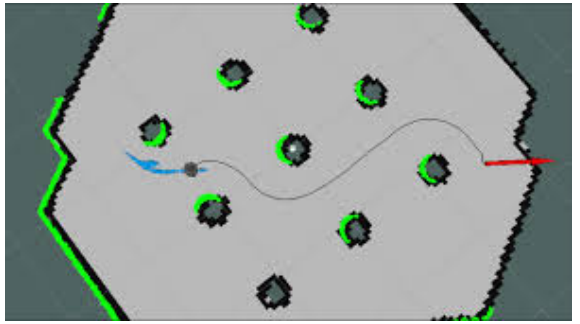
- A* planning on a graph
- RRT, RRT*





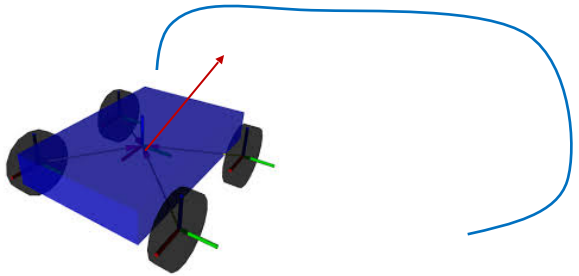
Trajectory optimization:

- Optimal control
- Numerical solvers



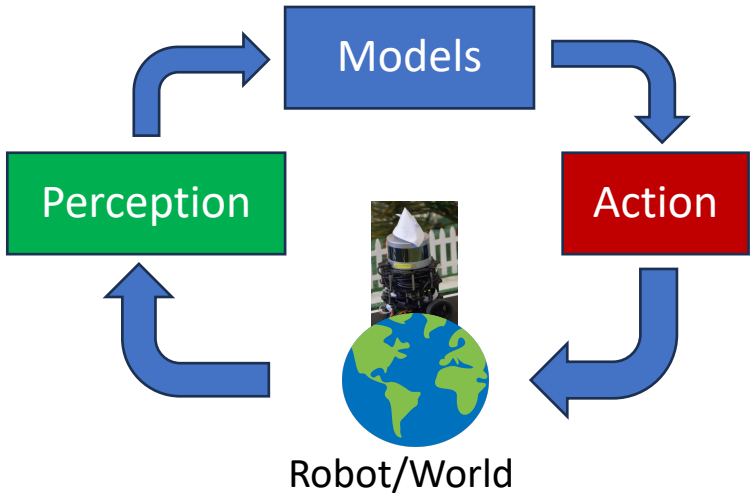
Trajectory following:

- Kinematic/dynamic models
- PID control
- LQR, gain scheduled LQR



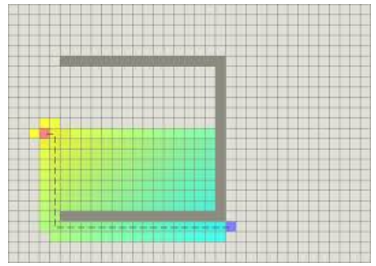
Action: Planning and Control Stack

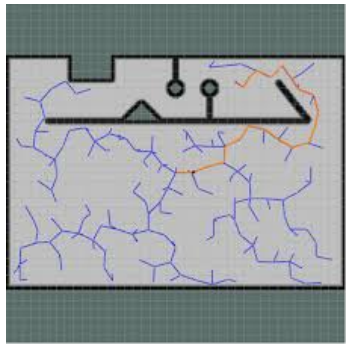
Use the models to move the robot



Path planning:

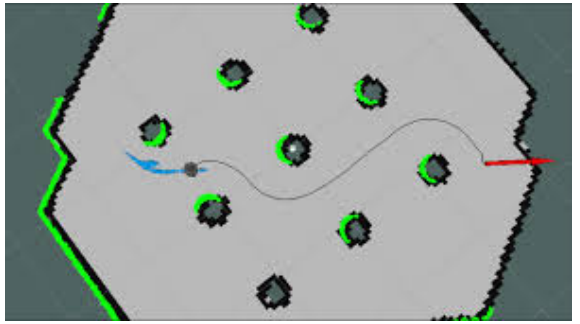
- A* planning on a graph
- RRT, RRT*





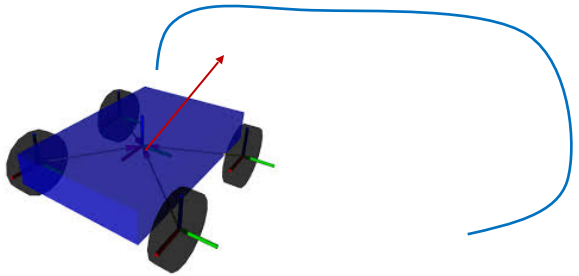
Trajectory optimization:

- Optimal control
- Numerical solvers



Trajectory following:

- Kinematic/dynamic models
- PID control
- LQR, gain scheduled LQR



Path Planning

Path Planning

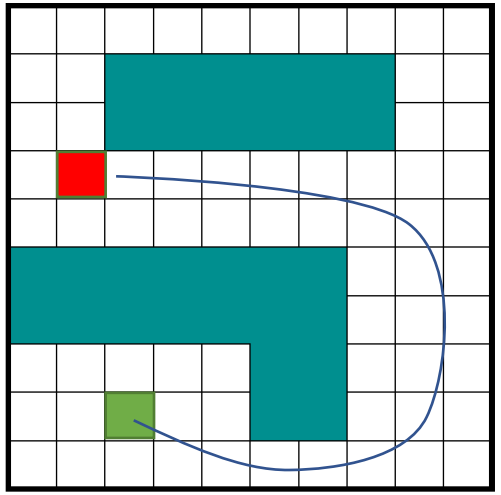
Path Planning

Map representation

Path Planning

Map representation

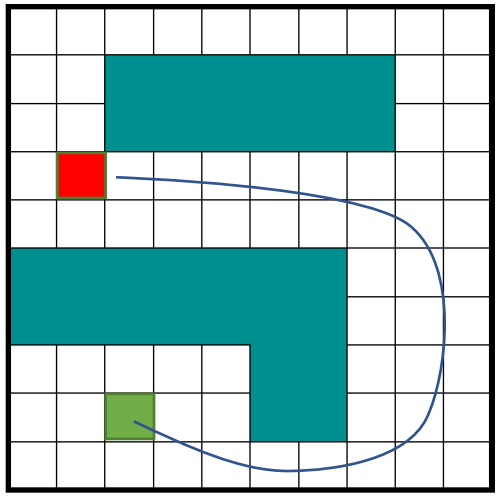
Graph search planning, A*



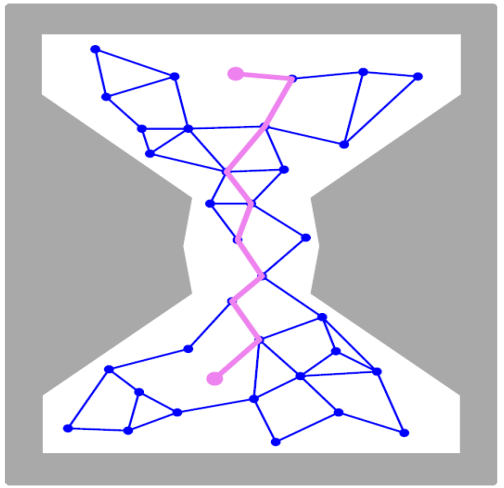
Path Planning

Map representation

Graph search planning, A*



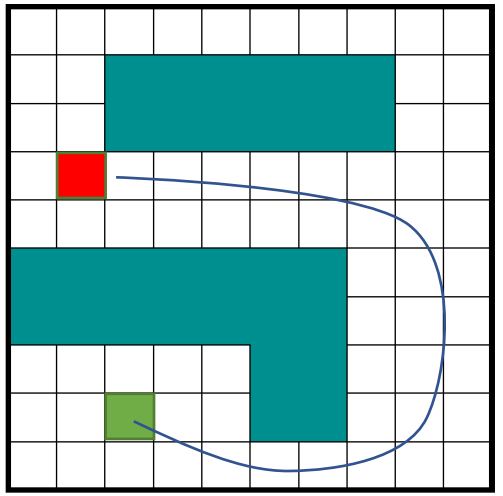
Planning on random graph, RRT*, PRM



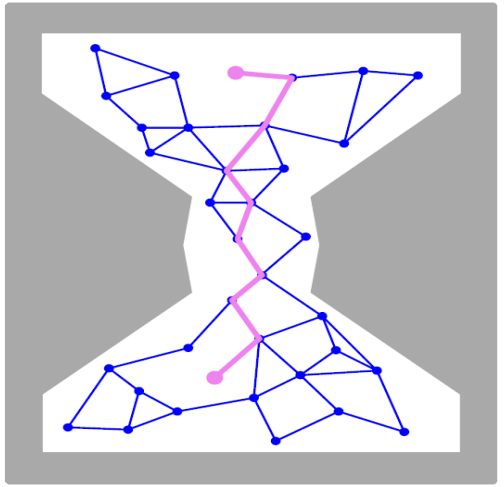
Path Planning

Map representation

Graph search planning, A*



Planning on random graph, RRT*, PRM



Goal: Find a path so the robot doesn't hit things!

Sampling-based planning

Challenges of explicit C-space computation motivated the development of sampling-based approaches

- Abandon the idea of explicitly characterizing C_{free} and C_{obs}
- Instead, capture the structure of C by random sampling
- Use a black-box component (collision checker) to determine which random configurations lie in C_{free}
- Use such a probing scheme to build a roadmap (a graph of possible paths) and then plan a path

Probabilistic roadmaps (PRM)

A multi-query planner, which generates a roadmap (**graph**) G, embedded in the free space

Preprocessing step:

1. Sample a collection of n configurations X_n; discard configurations leading to collisions
2. Draw an edge between each pair of samples $x, x' \in X_n$ such that $\|x - x'\| \leq r$
3. And straight-line path between x and x' is collision free

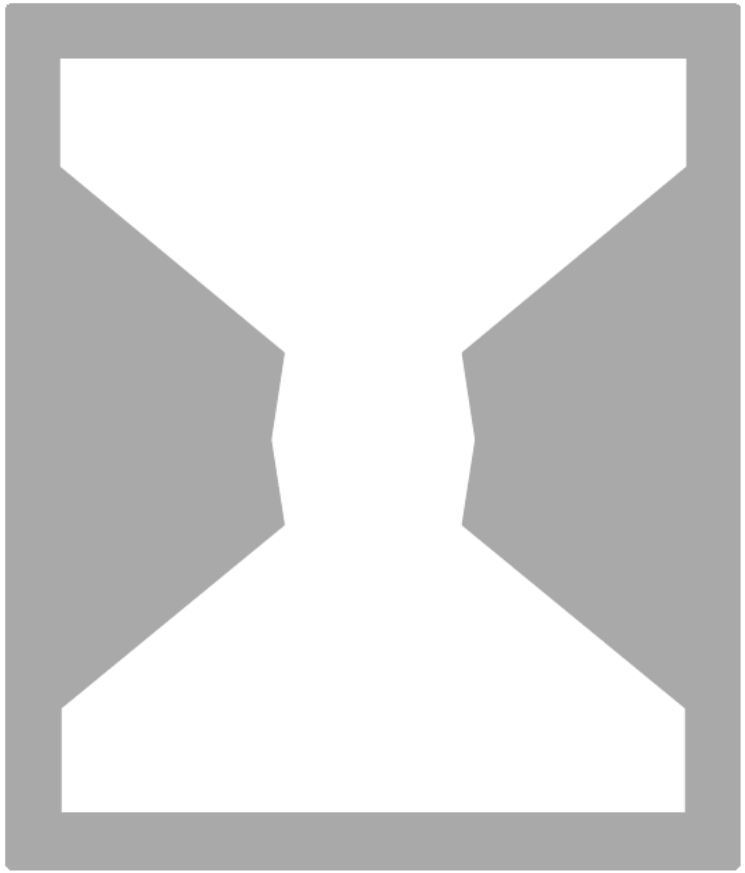
Given a query $s, t \in C_{free}$, connect them to G and find a path on the

Probabilistic roadmaps (PRM)

A multi-query planner, which generates a roadmap (**graph**) G, embedded in the free space

Preprocessing step:

1. Sample a collection of n configurations X_n; discard configurations leading to collisions
2. Draw an edge between each pair of samples $x, x' \in X_n$ such that $\|x - x'\| \leq r$
3. And straight-line path between x and x' is collision free



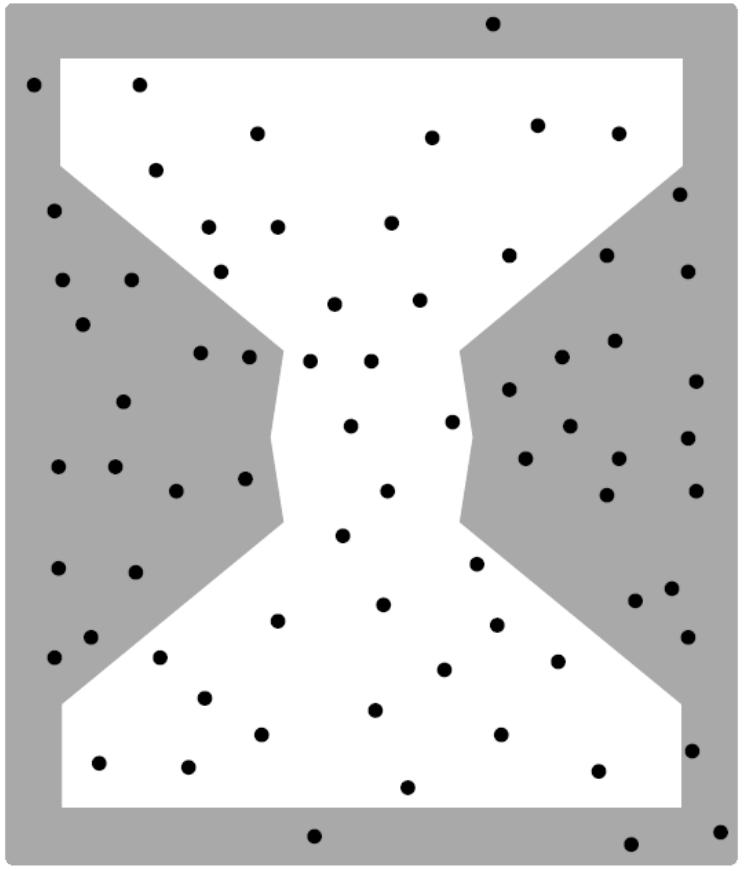
Given a query $s, t \in C_{free}$, connect them to G and find a path on the

Probabilistic roadmaps (PRM)

A multi-query planner, which generates a roadmap (**graph**) G, embedded in the free space

Preprocessing step:

1. Sample a collection of n configurations X_n; discard configurations leading to collisions
2. Draw an edge between each pair of samples $x, x' \in X_n$ such that $\|x - x'\| \leq r$
3. And straight-line path between x and x' is collision free



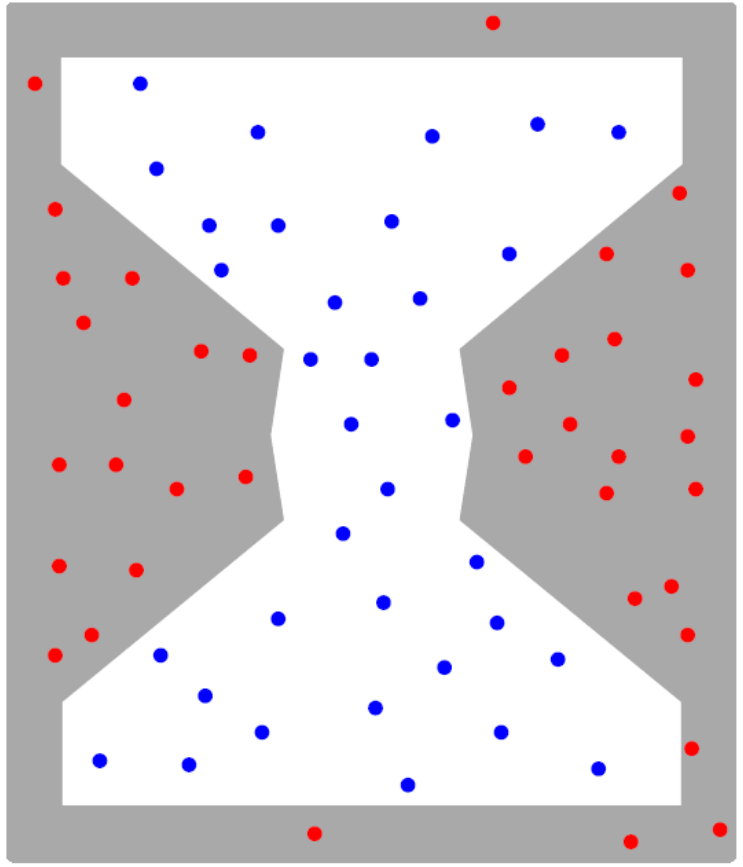
Given a query $s, t \in C_{free}$, connect them to G and find a path on the

Probabilistic roadmaps (PRM)

A multi-query planner, which generates a roadmap (**graph**) G, embedded in the free space

Preprocessing step:

1. Sample a collection of n configurations X_n; discard configurations leading to collisions
2. Draw an edge between each pair of samples $x, x' \in X_n$ such that $\|x - x'\| \leq r$
3. And straight-line path between x and x' is collision free



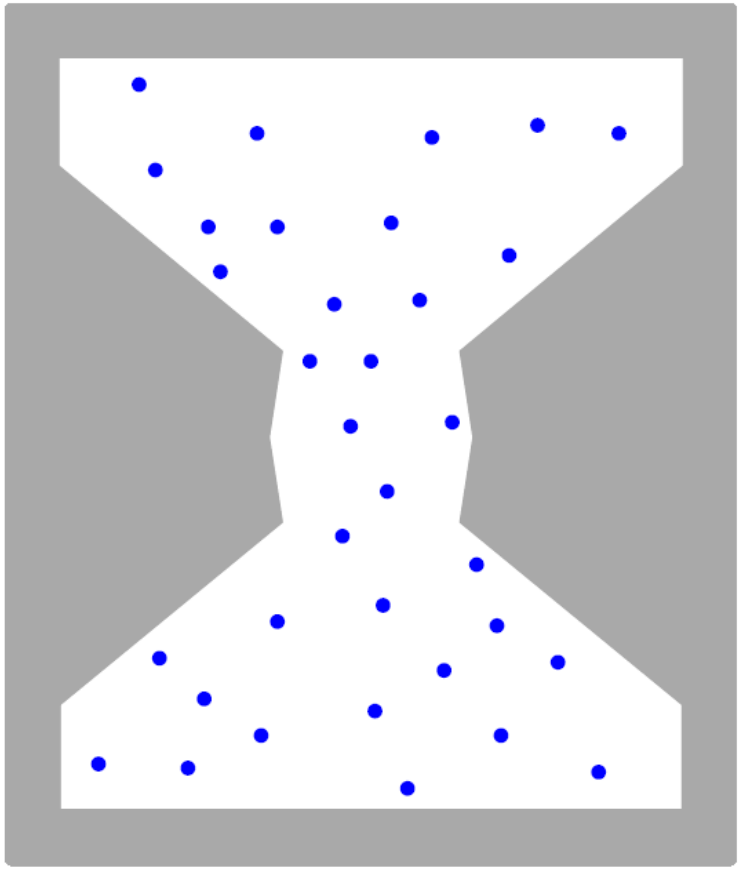
Given a query $s, t \in C_{free}$, connect them to G and find a path on the

Probabilistic roadmaps (PRM)

A multi-query planner, which generates a roadmap (**graph**) G, embedded in the free space

Preprocessing step:

1. Sample a collection of n configurations X_n; discard configurations leading to collisions
2. Draw an edge between each pair of samples $x, x' \in X_n$ such that $\|x - x'\| \leq r$
3. And straight-line path between x and x' is collision free



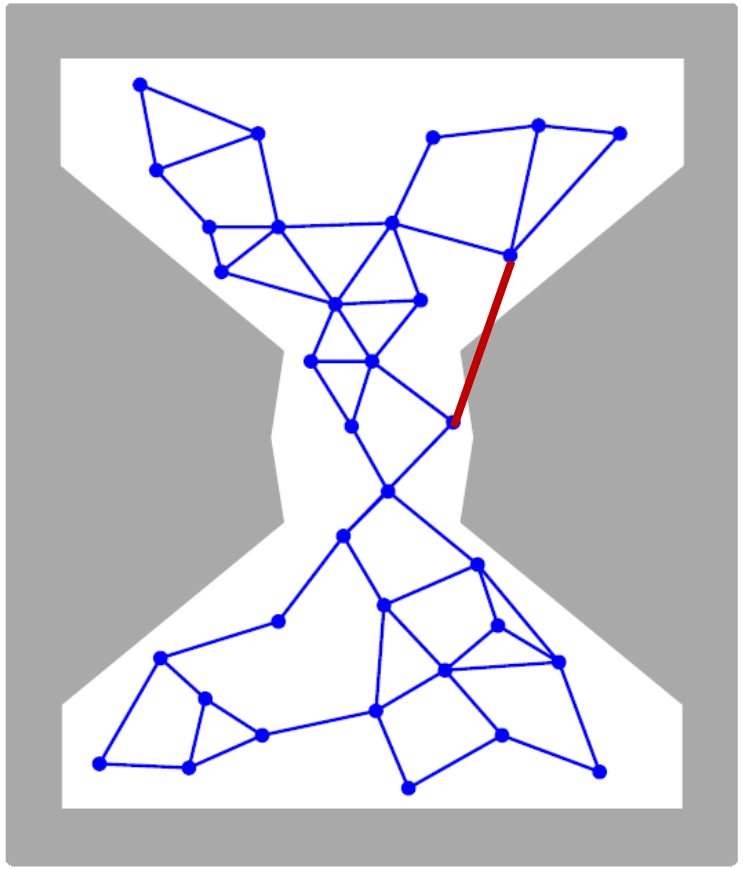
Given a query $s, t \in C_{free}$, connect them to G and find a path on the

Probabilistic roadmaps (PRM)

A multi-query planner, which generates a roadmap (**graph**) G, embedded in the free space

Preprocessing step:

1. Sample a collection of n configurations X_n; discard configurations leading to collisions
2. Draw an edge between each pair of samples $x, x' \in X_n$ such that $\|x - x'\| \leq r$
3. And straight-line path between x and x' is collision free



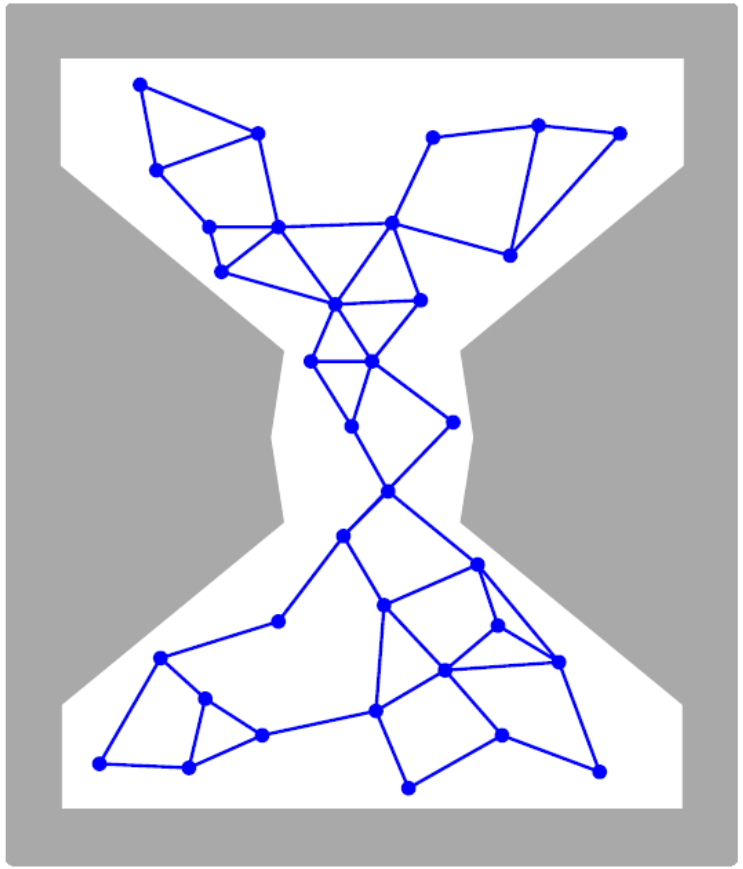
Given a query $s, t \in C_{free}$, connect them to G and find a path on the

Probabilistic roadmaps (PRM)

A multi-query planner, which generates a roadmap (**graph**) G, embedded in the free space

Preprocessing step:

1. Sample a collection of n configurations X_n; discard configurations leading to collisions
2. Draw an edge between each pair of samples $x, x' \in X_n$ such that $\|x - x'\| \leq r$
3. And straight-line path between x and x' is collision free



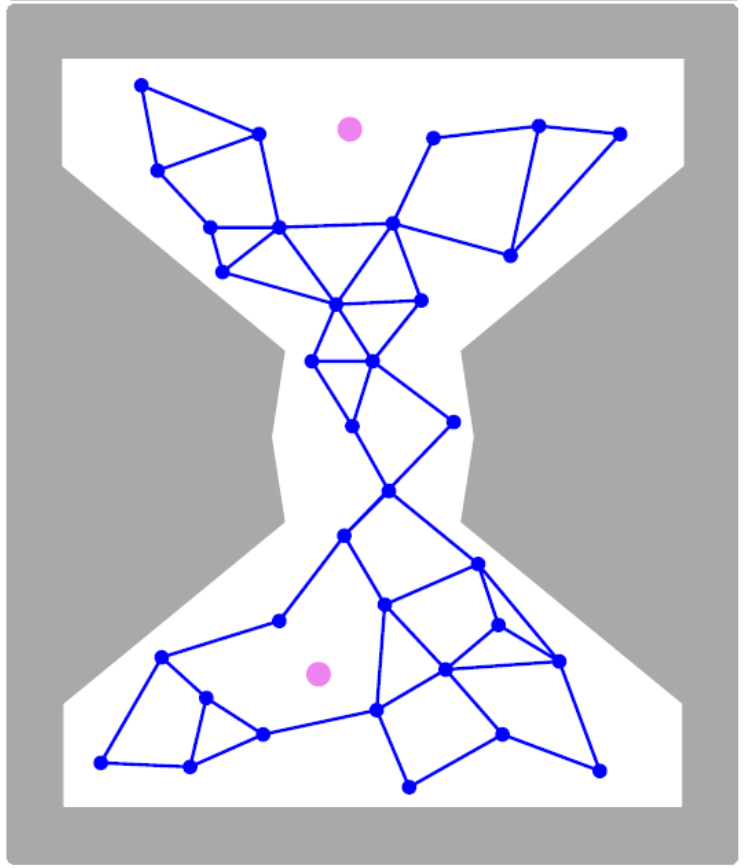
Given a query $s, t \in C_{free}$, connect them to G and find a path on the

Probabilistic roadmaps (PRM)

A multi-query planner, which generates a roadmap (**graph**) G, embedded in the free space

Preprocessing step:

1. Sample a collection of n configurations X_n; discard configurations leading to collisions
2. Draw an edge between each pair of samples $x, x' \in X_n$ such that $\|x - x'\| \leq r$
3. And straight-line path between x and x' is collision free



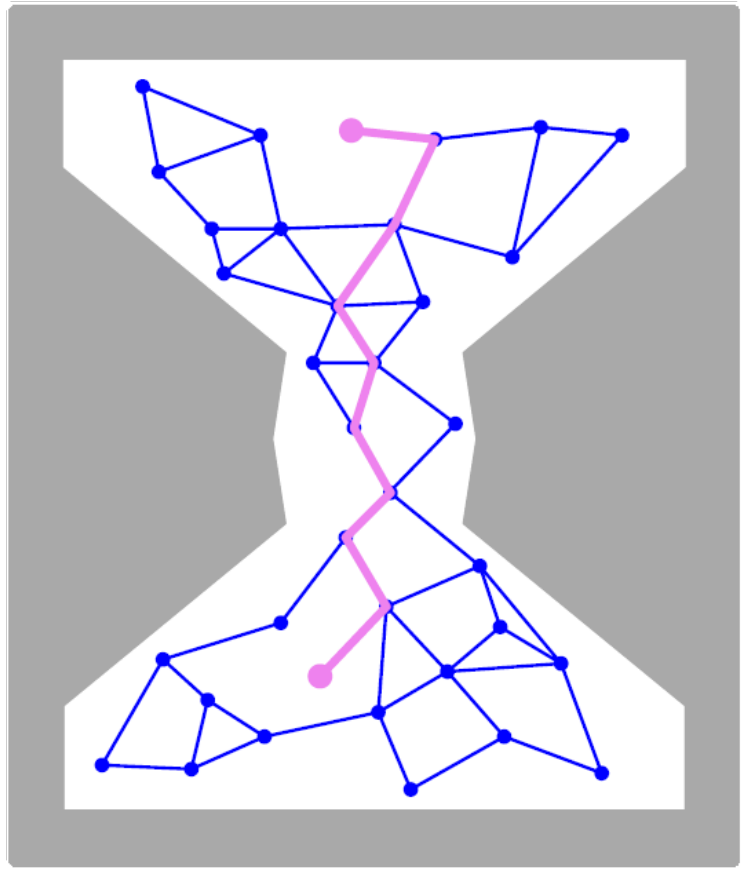
Given a query $s, t \in C_{free}$, connect them to G and find a path on the

Probabilistic roadmaps (PRM)

A multi-query planner, which generates a roadmap (**graph**) G, embedded in the free space

Preprocessing step:

1. Sample a collection of n configurations X_n; discard configurations leading to collisions
2. Draw an edge between each pair of samples $x, x' \in X_n$ such that $\|x - x'\| \leq r$
3. And straight-line path between x and x' is collision free



Given a query $s, t \in C_{free}$, connect them to G and find a path on the

Rapidly-exploring random trees (RRT)

A single-query planner, which grows a **tree** $T = (V, E)$, rooted at the start configuration s, embedded in C_{free}

Algorithm works in n iterations:

1. Sample configuration x_{rand}
2. Find nearest vertex x_{near} in T to x_{rand}
3. Generate configuration x_{new} in direction of x_{rand} from x_{near}, such that $\|x_{near} - x_{new}\| \leq \eta$ and $\{x \mid \alpha x_{near} + (1-\alpha)x_{new}, \alpha \in [0,1]\} \in C_{free}$
4. Update tree: $T = (V \cup x_{new}, E \cup (x_{near}, x_{new}))$

Occasionally, set x_{rand} to be the target vertex t; terminate when $x_{new} = t$

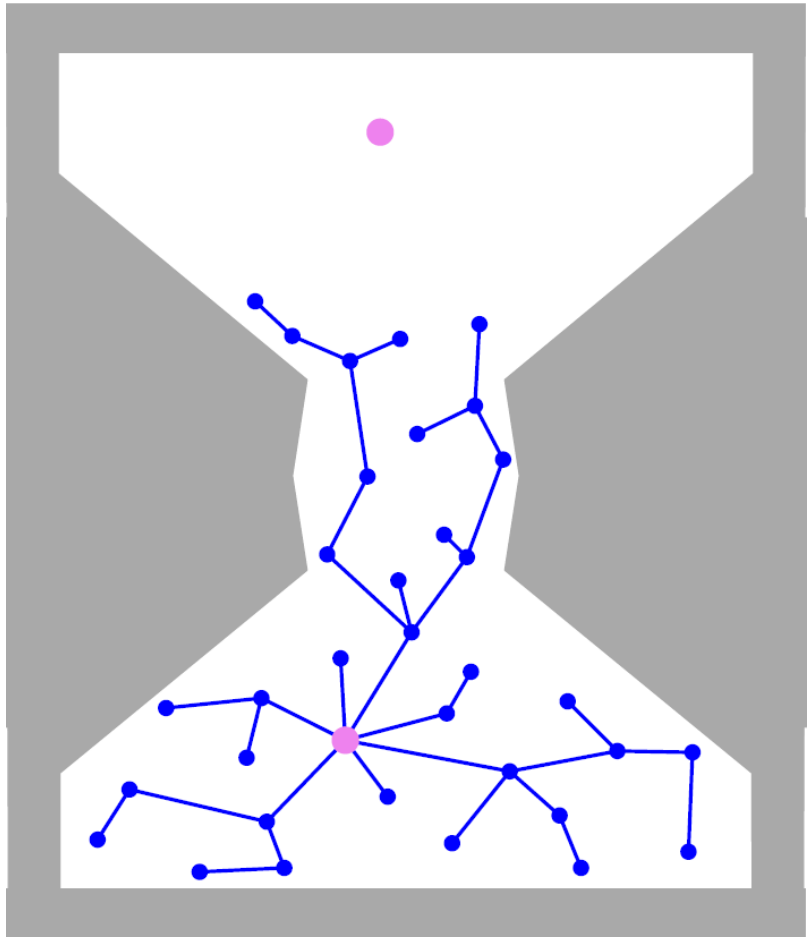
Rapidly-exploring random trees (RRT)

A single-query planner, which grows a **tree** $T = (V, E)$, rooted at the start configuration s, embedded in C_{free}

Algorithm works in n iterations:

1. Sample configuration x_{rand}
2. Find nearest vertex x_{near} in T to x_{rand}
3. Generate configuration x_{new} in direction of x_{rand} from x_{near}, such that $\|x_{near} - x_{new}\| \leq \eta$ and $\{x \mid \alpha x_{near} + (1-\alpha)x_{new}, \alpha \in [0,1]\} \in C_{free}$
4. Update tree: $T = (V \cup x_{new}, E \cup (x_{near}, x_{new}))$

Occasionally, set x_{rand} to be the target vertex t; terminate when $x_{new} = t$



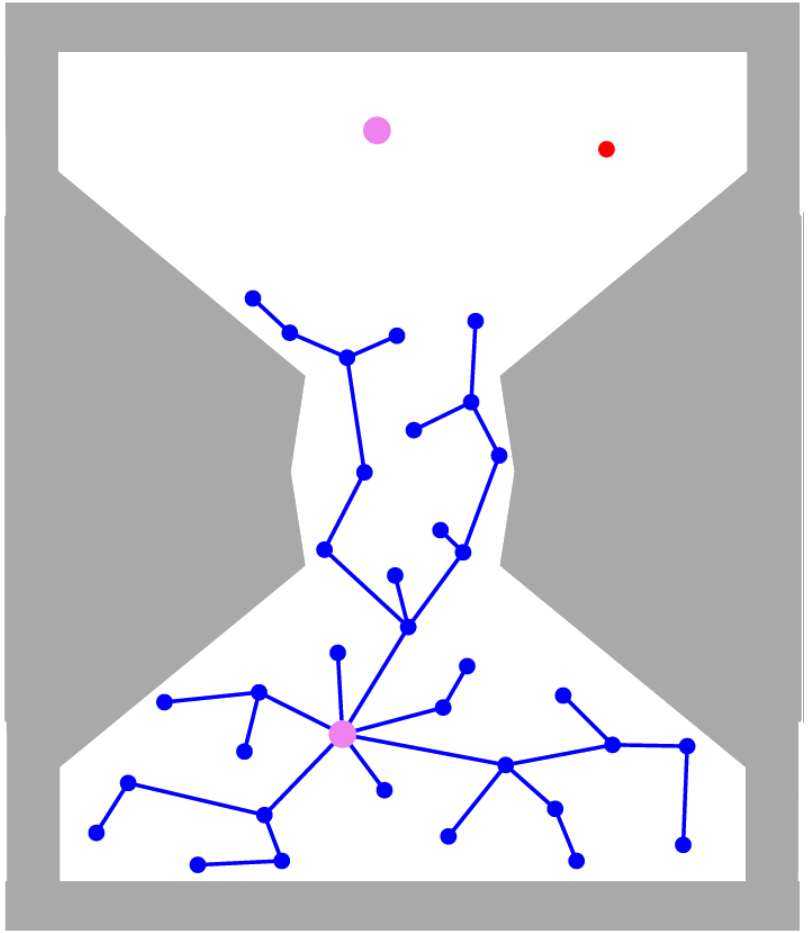
Rapidly-exploring random trees (RRT)

A single-query planner, which grows a **tree** $T = (V, E)$, rooted at the start configuration s, embedded in C_{free}

Algorithm works in n iterations:

1. Sample configuration x_{rand}
2. Find nearest vertex x_{near} in T to x_{rand}
3. Generate configuration x_{new} in direction of x_{rand} from x_{near}, such that $\|x_{near} - x_{new}\| \leq \eta$ and $\{x \mid \alpha x_{near} + (1-\alpha)x_{new}, \alpha \in [0,1]\} \in C_{free}$
4. Update tree: $T = (V \cup x_{new}, E \cup (x_{near}, x_{new}))$

Occasionally, set x_{rand} to be the target vertex t; terminate when $x_{new} = t$



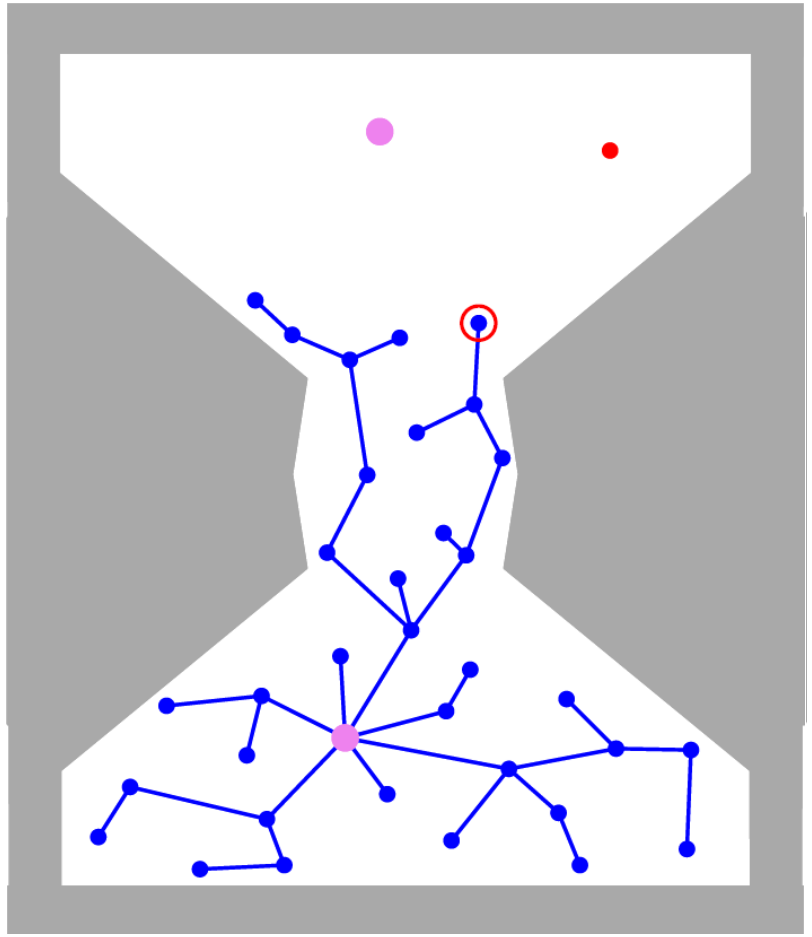
Rapidly-exploring random trees (RRT)

A single-query planner, which grows a **tree** $T = (V, E)$, rooted at the start configuration s, embedded in C_{free}

Algorithm works in n iterations:

1. Sample configuration x_{rand}
2. Find nearest vertex x_{near} in T to x_{rand}
3. Generate configuration x_{new} in direction of x_{rand} from x_{near}, such that $\|x_{near} - x_{new}\| \leq \eta$ and $\{x \mid \alpha x_{near} + (1-\alpha)x_{new}, \alpha \in [0,1]\} \in C_{free}$
4. Update tree: $T = (V \cup x_{new}, E \cup (x_{near}, x_{new}))$

Occasionally, set x_{rand} to be the target vertex t; terminate when $x_{new} = t$



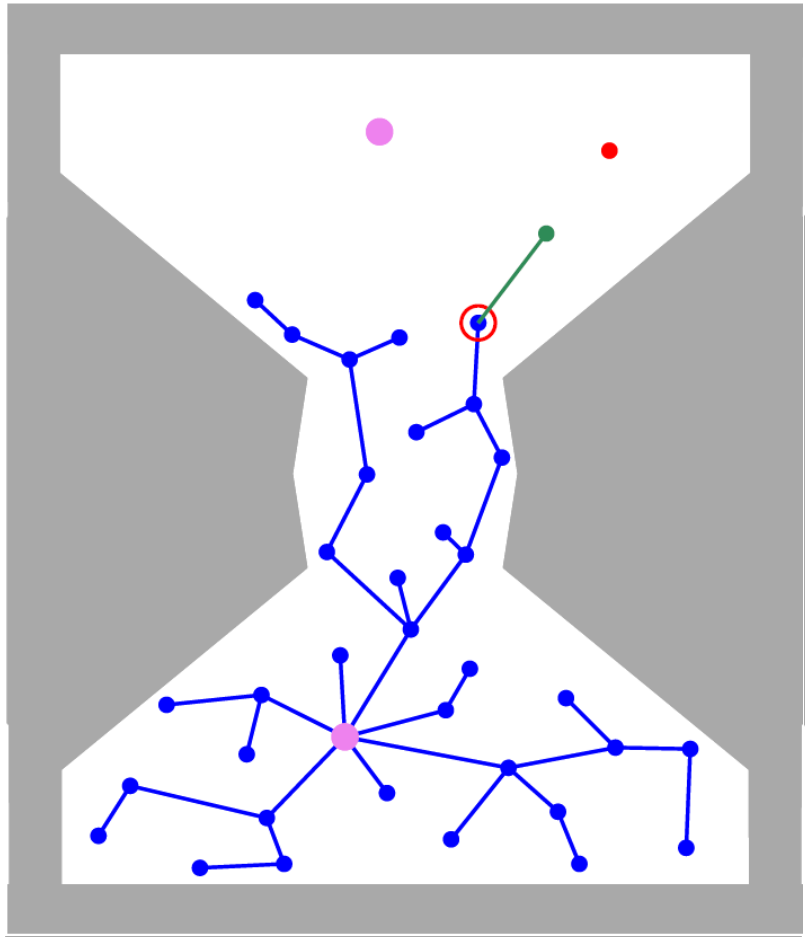
Rapidly-exploring random trees (RRT)

A single-query planner, which grows a **tree** $T = (V, E)$, rooted at the start configuration s, embedded in C_{free}

Algorithm works in n iterations:

1. Sample configuration x_{rand}
2. Find nearest vertex x_{near} in T to x_{rand}
3. Generate configuration x_{new} in direction of x_{rand} from x_{near}, such that $\|x_{near} - x_{new}\| \leq \eta$ and $\{x \mid \alpha x_{near} + (1-\alpha)x_{new}, \alpha \in [0,1]\} \in C_{free}$
4. Update tree: $T = (V \cup x_{new}, E \cup (x_{near}, x_{new}))$

Occasionally, set x_{rand} to be the target vertex t; terminate when $x_{new} = t$



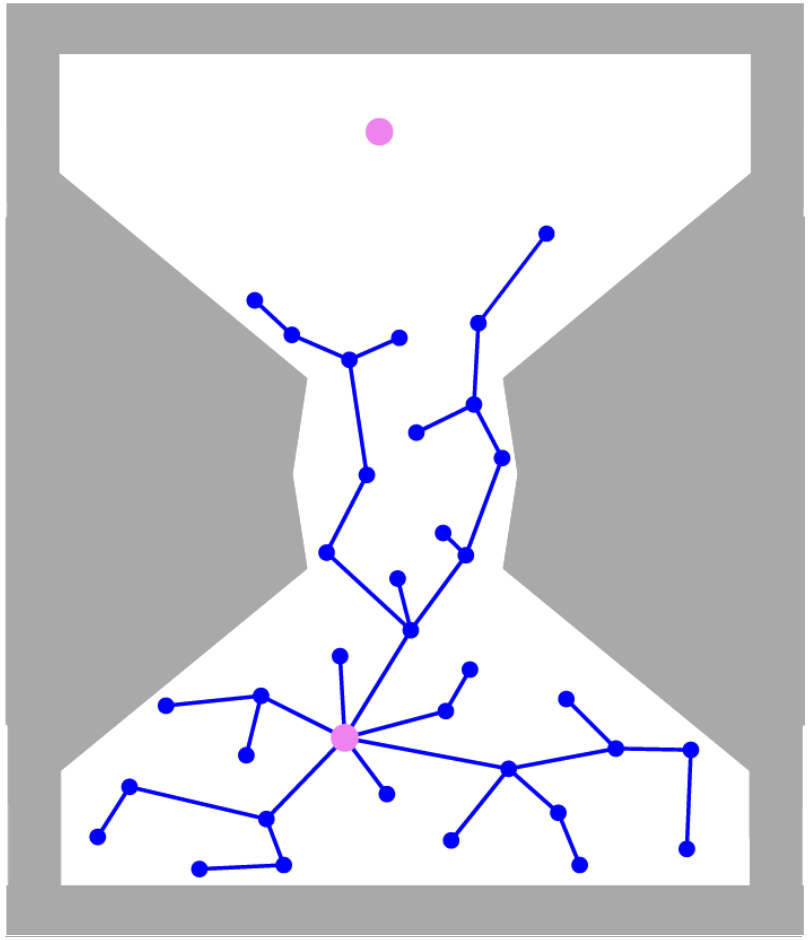
Rapidly-exploring random trees (RRT)

A single-query planner, which grows a **tree** $T = (V, E)$, rooted at the start configuration s, embedded in C_{free}

Algorithm works in n iterations:

1. Sample configuration x_{rand}
2. Find nearest vertex x_{near} in T to x_{rand}
3. Generate configuration x_{new} in direction of x_{rand} from x_{near}, such that $\|x_{near} - x_{new}\| \leq \eta$ and $\{x \mid \alpha x_{near} + (1-\alpha)x_{new}, \alpha \in [0,1]\} \in C_{free}$
4. Update tree: $T = (V \cup x_{new}, E \cup (x_{near}, x_{new}))$

Occasionally, set x_{rand} to be the target vertex t; terminate when $x_{new} = t$



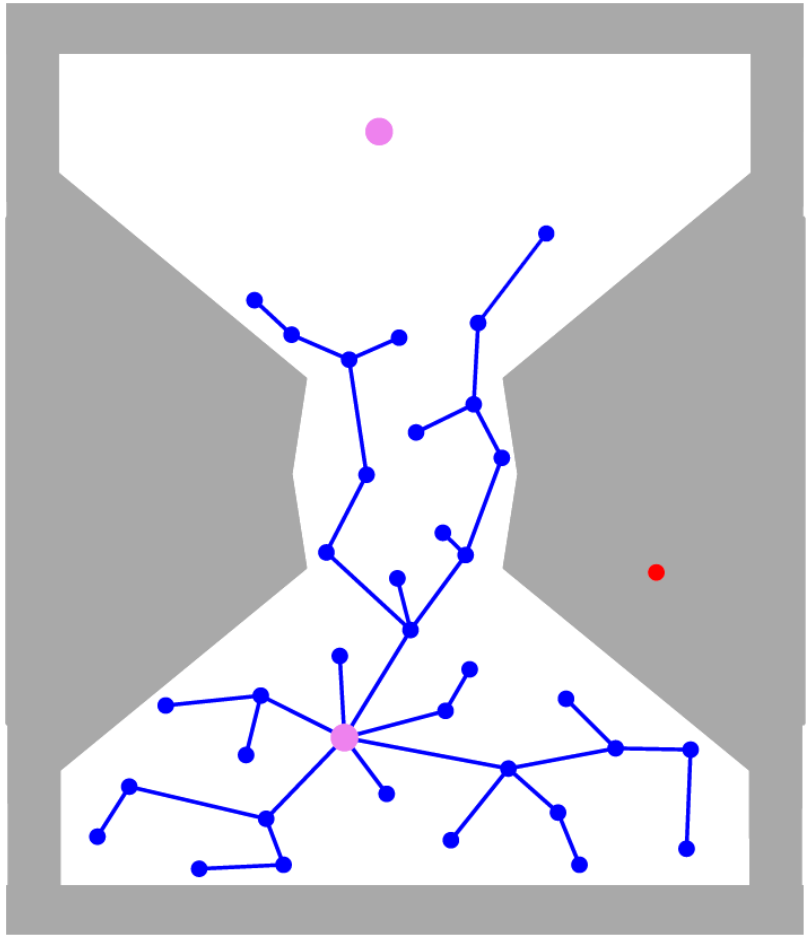
Rapidly-exploring random trees (RRT)

A single-query planner, which grows a **tree** $T = (V, E)$, rooted at the start configuration s, embedded in C_{free}

Algorithm works in n iterations:

1. Sample configuration x_{rand}
2. Find nearest vertex x_{near} in T to x_{rand}
3. Generate configuration x_{new} in direction of x_{rand} from x_{near}, such that $\|x_{near} - x_{new}\| \leq \eta$ and $\{x \mid \alpha x_{near} + (1-\alpha)x_{new}, \alpha \in [0,1]\} \in C_{free}$
4. Update tree: $T = (V \cup x_{new}, E \cup (x_{near}, x_{new}))$

Occasionally, set x_{rand} to be the target vertex t; terminate when $x_{new} = t$



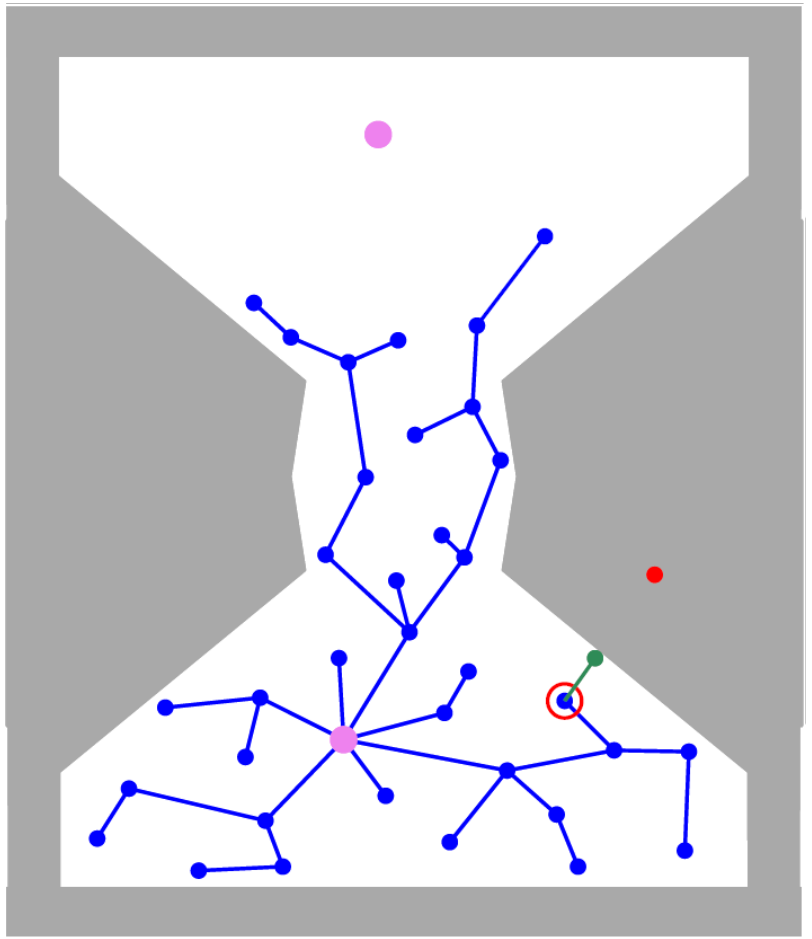
Rapidly-exploring random trees (RRT)

A single-query planner, which grows a **tree** $T = (V, E)$, rooted at the start configuration s, embedded in C_{free}

Algorithm works in n iterations:

1. Sample configuration x_{rand}
2. Find nearest vertex x_{near} in T to x_{rand}
3. Generate configuration x_{new} in direction of x_{rand} from x_{near}, such that $\|x_{near} - x_{new}\| \leq \eta$ and $\{x \mid \alpha x_{near} + (1-\alpha)x_{new}, \alpha \in [0,1]\} \in C_{free}$
4. Update tree: $T = (V \cup x_{new}, E \cup (x_{near}, x_{new}))$

Occasionally, set x_{rand} to be the target vertex t; terminate when $x_{new} = t$



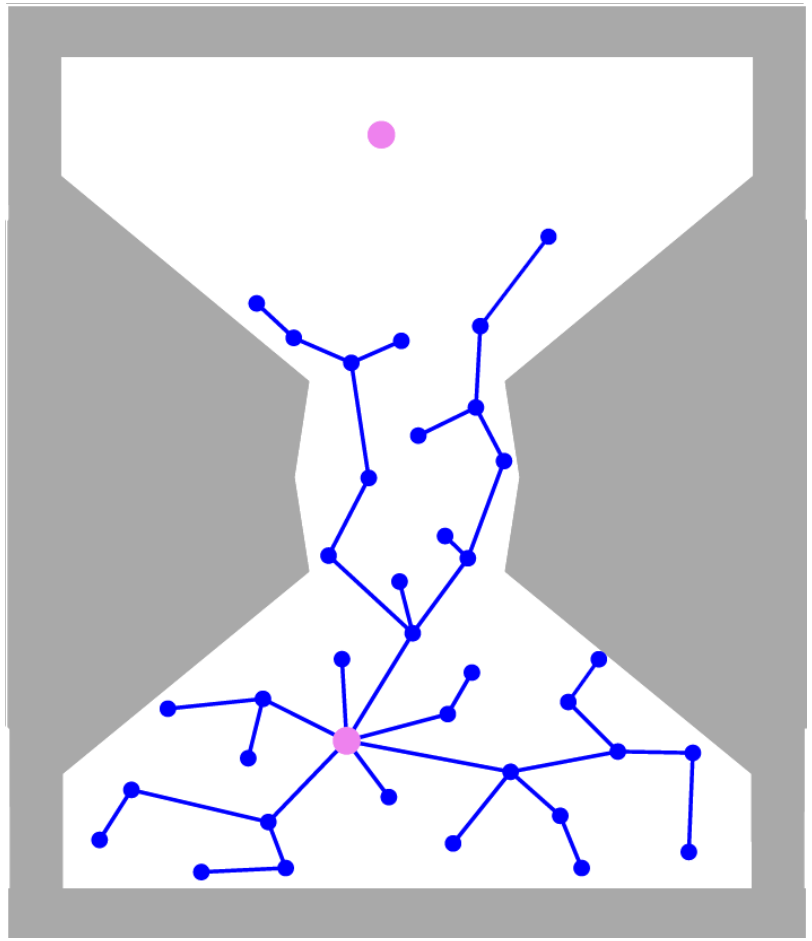
Rapidly-exploring random trees (RRT)

A single-query planner, which grows a **tree** $T = (V, E)$, rooted at the start configuration s, embedded in C_{free}

Algorithm works in n iterations:

1. Sample configuration x_{rand}
2. Find nearest vertex x_{near} in T to x_{rand}
3. Generate configuration x_{new} in direction of x_{rand} from x_{near}, such that $\|x_{near} - x_{new}\| \leq \eta$ and $\{x \mid \alpha x_{near} + (1-\alpha)x_{new}, \alpha \in [0,1]\} \in C_{free}$
4. Update tree: $T = (V \cup x_{new}, E \cup (x_{near}, x_{new}))$

Occasionally, set x_{rand} to be the target vertex t; terminate when $x_{new} = t$



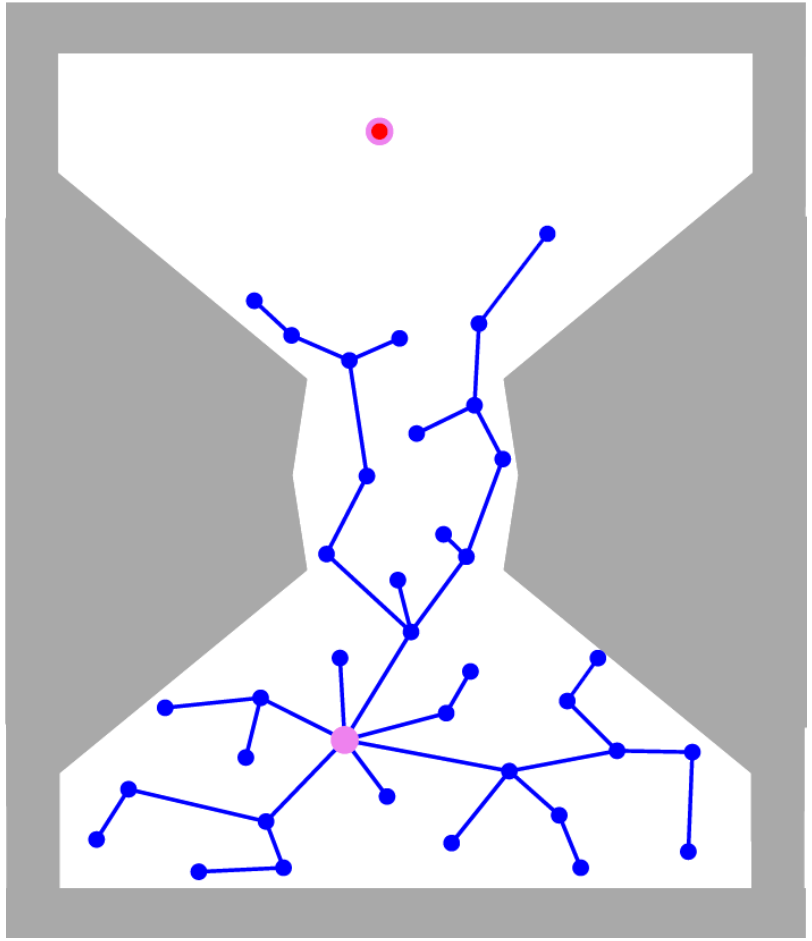
Rapidly-exploring random trees (RRT)

A single-query planner, which grows a **tree** $T = (V, E)$, rooted at the start configuration s, embedded in C_{free}

Algorithm works in n iterations:

1. Sample configuration x_{rand}
2. Find nearest vertex x_{near} in T to x_{rand}
3. Generate configuration x_{new} in direction of x_{rand} from x_{near}, such that $\|x_{near} - x_{new}\| \leq \eta$ and $\{x \mid \alpha x_{near} + (1-\alpha)x_{new}, \alpha \in [0,1]\} \in C_{free}$
4. Update tree: $T = (V \cup x_{new}, E \cup (x_{near}, x_{new}))$

Occasionally, set x_{rand} to be the target vertex t; terminate when $x_{new} = t$



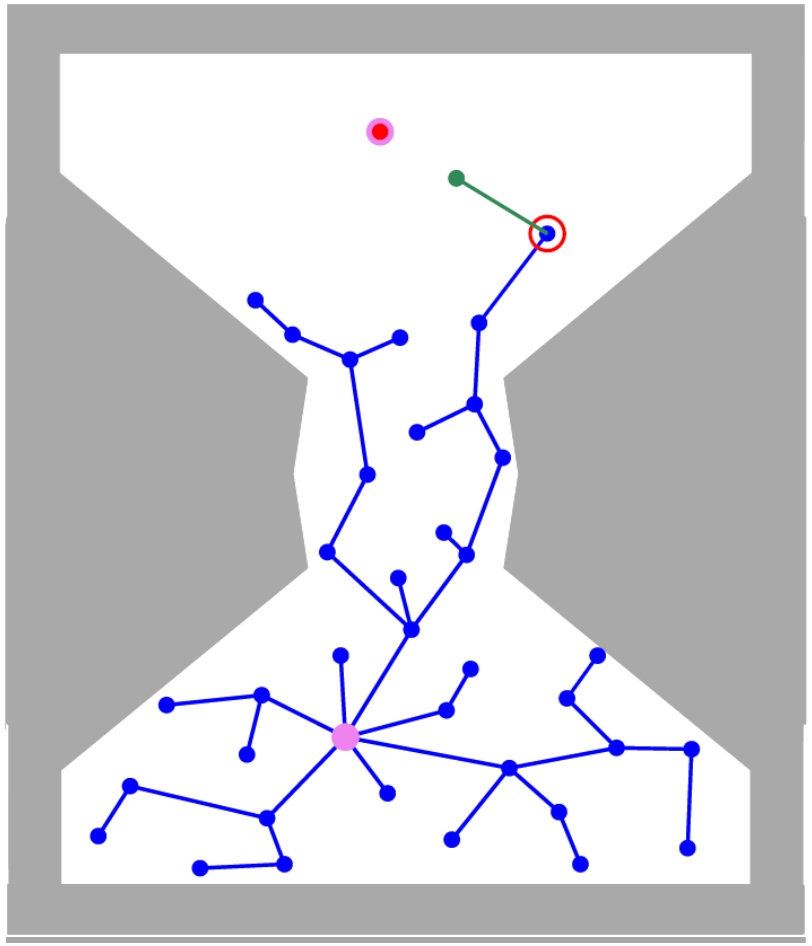
Rapidly-exploring random trees (RRT)

A single-query planner, which grows a **tree** $T = (V, E)$, rooted at the start configuration s, embedded in C_{free}

Algorithm works in n iterations:

1. Sample configuration x_{rand}
2. Find nearest vertex x_{near} in T to x_{rand}
3. Generate configuration x_{new} in direction of x_{rand} from x_{near}, such that $\|x_{near} - x_{new}\| \leq \eta$ and $\{x \mid \alpha x_{near} + (1-\alpha)x_{new}, \alpha \in [0,1]\} \in C_{free}$
4. Update tree: $T = (V \cup x_{new}, E \cup (x_{near}, x_{new}))$

Occasionally, set x_{rand} to be the target vertex t; terminate when $x_{new} = t$



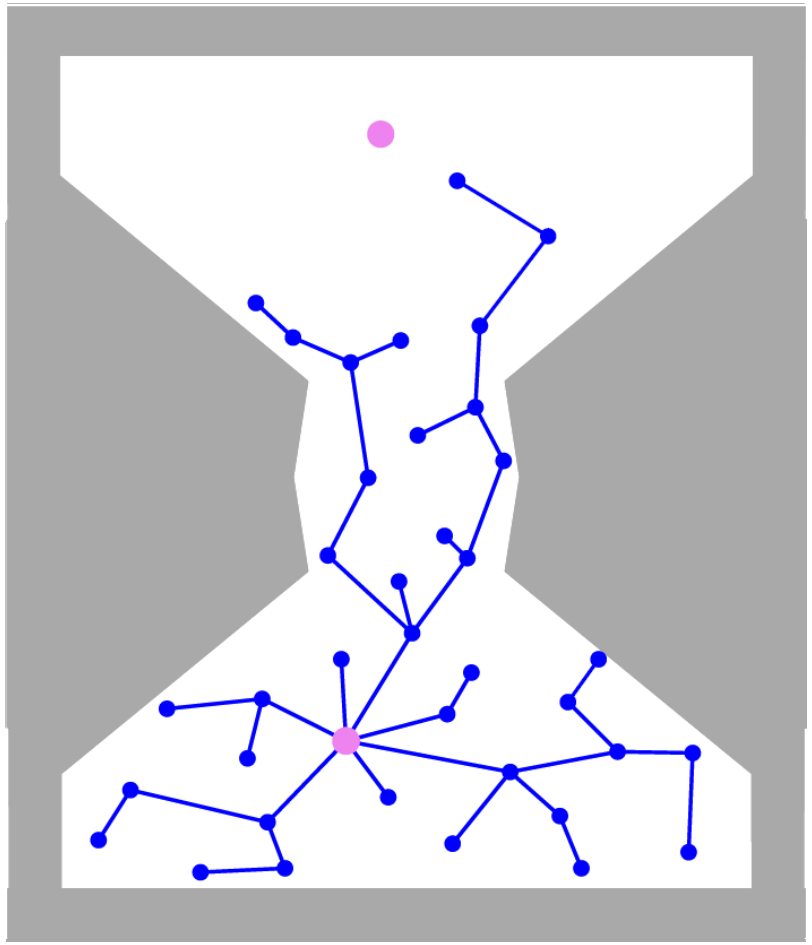
Rapidly-exploring random trees (RRT)

A single-query planner, which grows a **tree** $T = (V, E)$, rooted at the start configuration s, embedded in C_{free}

Algorithm works in n iterations:

1. Sample configuration x_{rand}
2. Find nearest vertex x_{near} in T to x_{rand}
3. Generate configuration x_{new} in direction of x_{rand} from x_{near}, such that $\|x_{near} - x_{new}\| \leq \eta$ and $\{x \mid \alpha x_{near} + (1-\alpha)x_{new}, \alpha \in [0,1]\} \in C_{free}$
4. Update tree: $T = (V \cup x_{new}, E \cup (x_{near}, x_{new}))$

Occasionally, set x_{rand} to be the target vertex t; terminate when $x_{new} = t$



Theoretical guarantees: probabilistic completeness

Question: how large should the number of samples n be? We can say something about the asymptotic behavior:

Kavraki et al. 96: For PRM, with $r = \text{const}$, the probability of finding a path if one exists approaches 1 as $n \to \infty$.

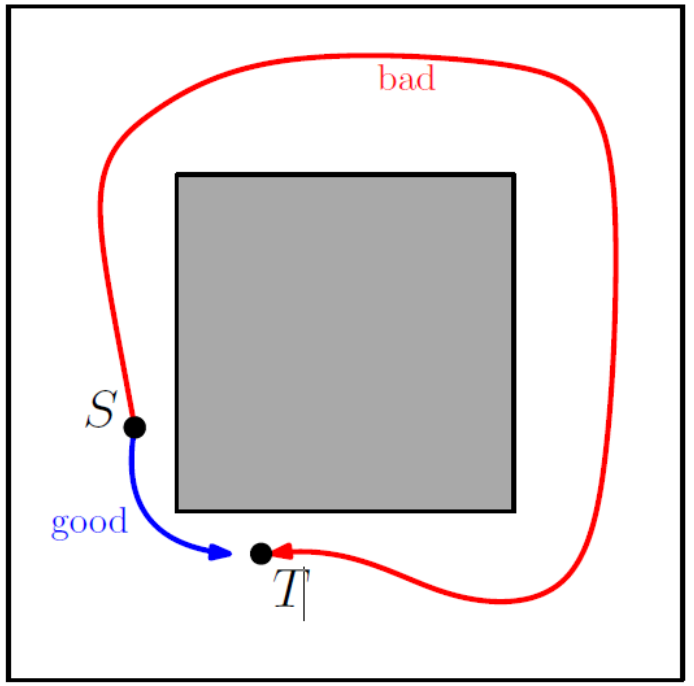
LaValle, 98; Kleinbort et al., 18: For RRT, the probability of finding a path if one exists approaches 1 as $n \to \infty$

* hypercube $[0,1]^d$, with $2 \leq d < \infty$

Theoretical guarantees: quality

Question: what can be said about the quality of the returned solution for PRM and RRT, in terms of length, energy, etc.?

- Nechushtan et al. (2011) and Karaman and Frazzoli (2011) proved that RRT can produce arbitrarily-bad paths with non-negligible probability: for example, RRT might prefer to take the long (red) way.
- This is because it does not **revise** connections with more efficient choices that might be found later (recall Dijkstra's, A*).

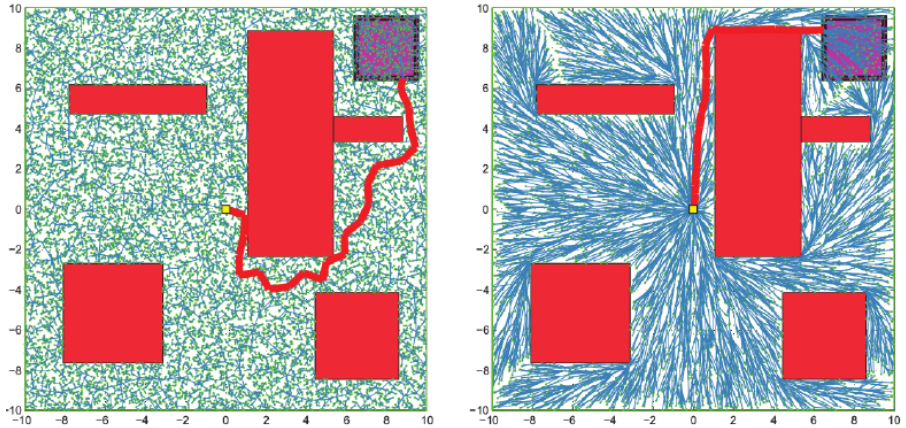


Theoretical guarantees: quality

Karaman and Frazzoli in 2011 provided the first rigorous study of optimality in sampling-based planners:

Theorem: The cost of the solution returned by PRM converges, as $n \to \infty$, to the optimum, when $r_n = \gamma\left(\frac{\log n}{n}\right)^{\frac{1}{d}}$, where γ only depends on d

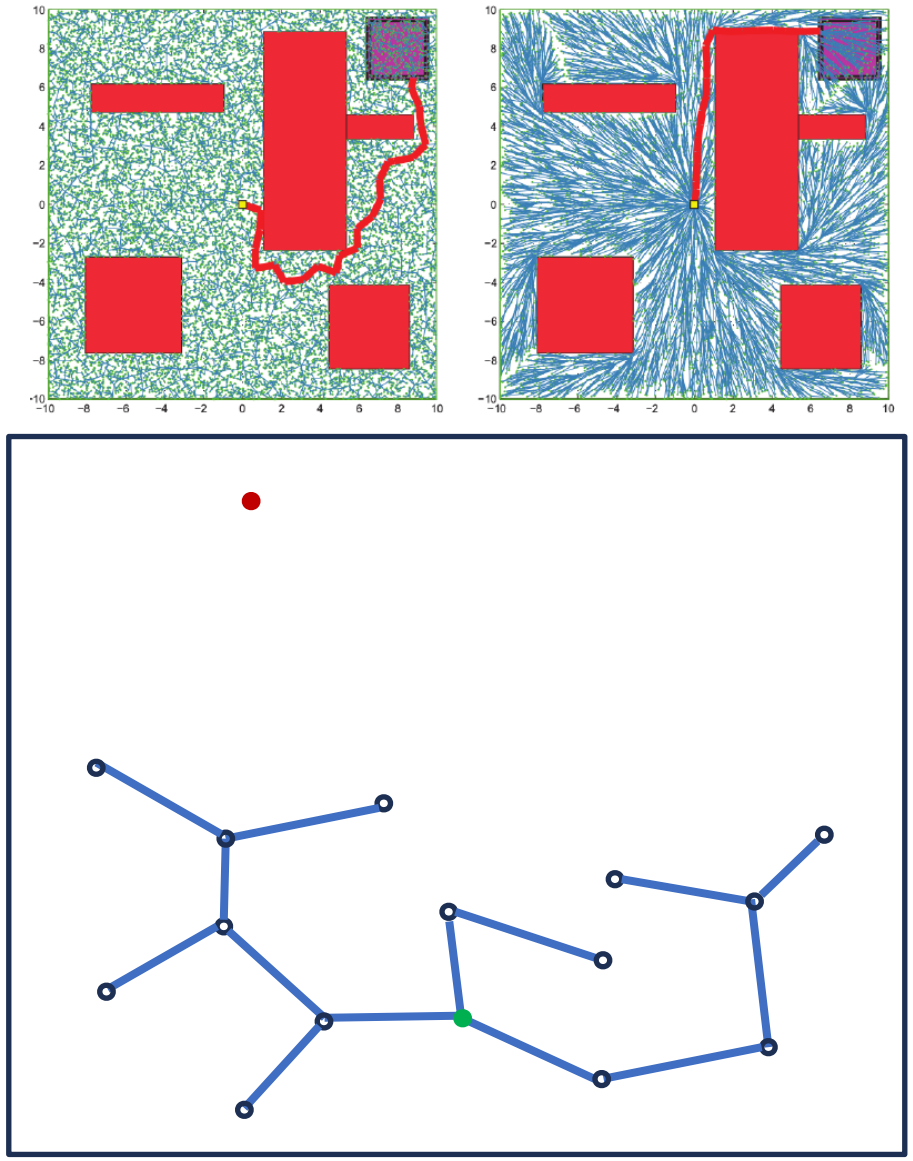
- KF11 also introduced an asymptotically optimal variant of RRT called RRT* (right)
- Introduced **re-wiring,** another name for label correcting, just like Dijkstra's/A*.



RRT* Re-wiring

- After connecting vertex x_{new} to the tree, check alternative connections to all potential parents in ball radius r_n
- If a lower cost path found through different parent, add edge to new parent, delete edge to old parent
- If a lower cost path found through x_{new} as parent, add new/delete old
- r_n shrinks as n grows, according to

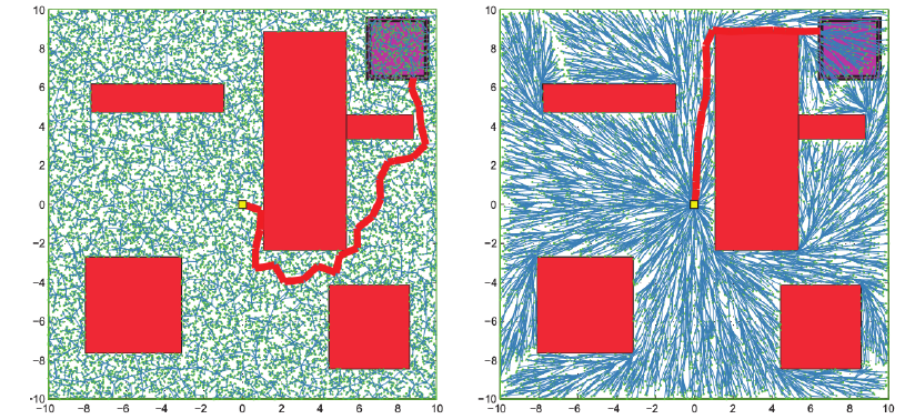
$$r_n = \gamma \left(\frac{\log n}{n} \right)^{\frac{1}{d}}$$

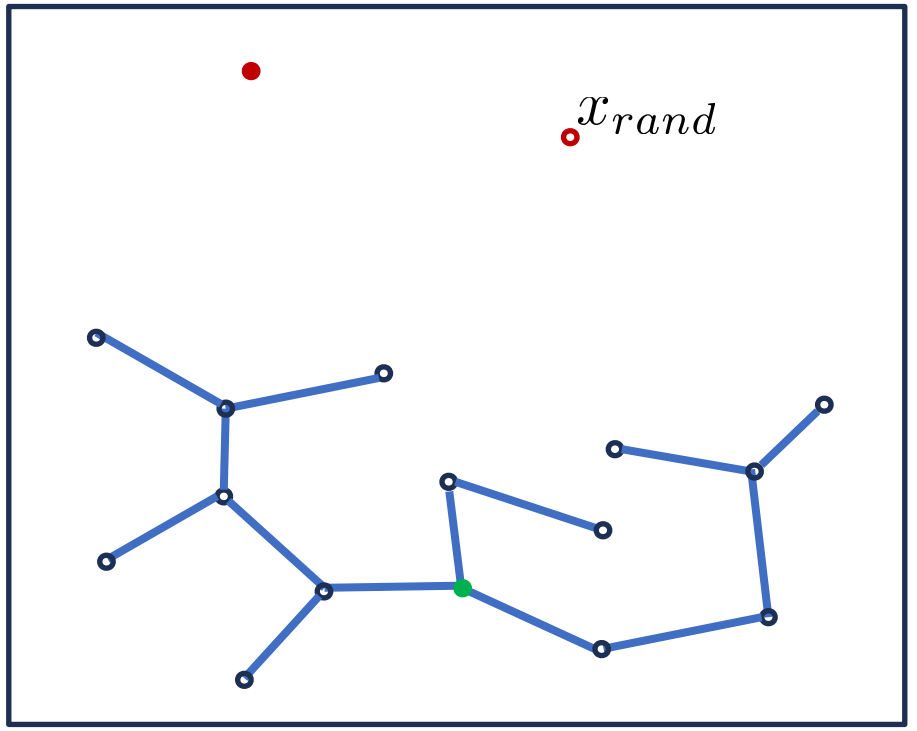


RRT* Re-wiring

- After connecting vertex x_{new} to the tree, check alternative connections to all potential parents in ball radius r_n
- If a lower cost path found through different parent, add edge to new parent, delete edge to old parent
- If a lower cost path found through x_{new} as parent, add new/delete old
- r_n shrinks as n grows, according to

$$r_n = \gamma \left(\frac{\log n}{n} \right)^{\frac{1}{d}}$$

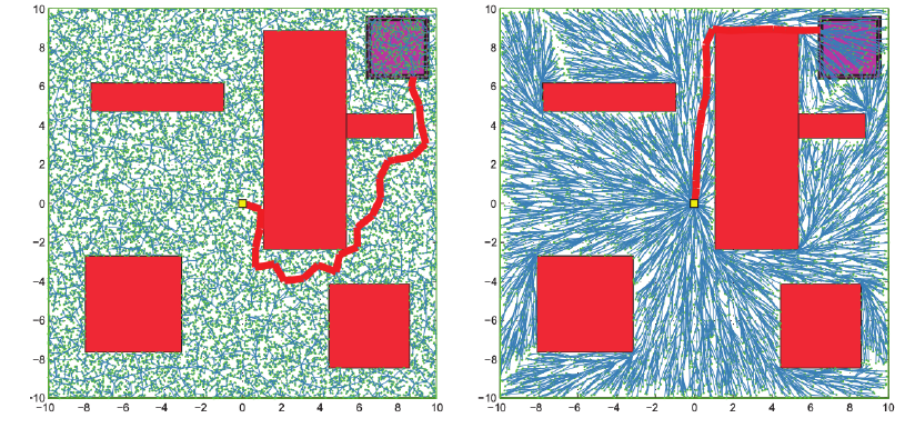


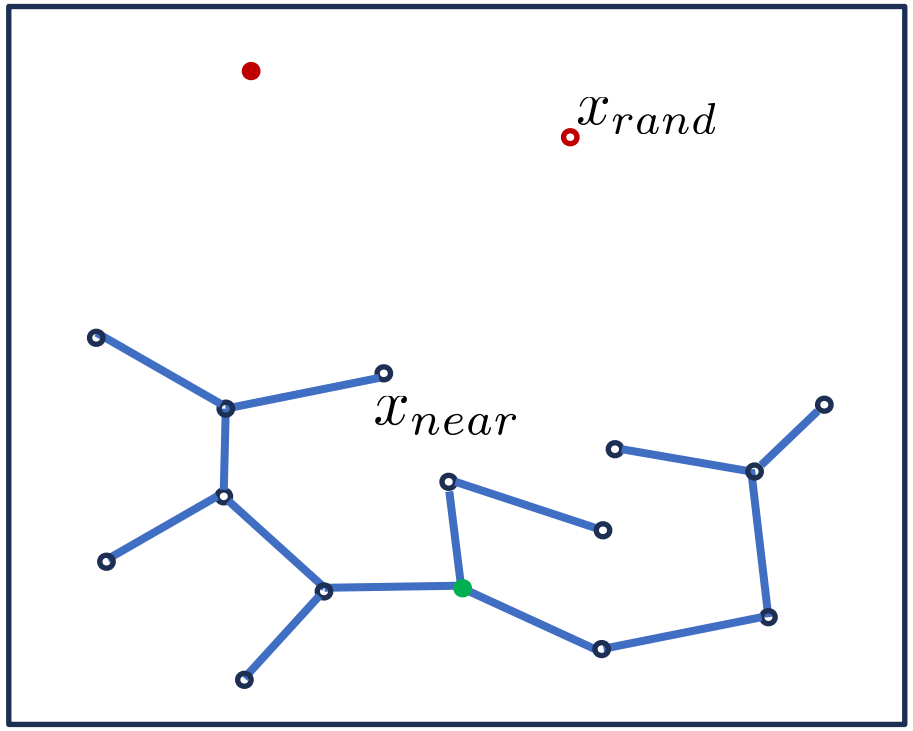


RRT* Re-wiring

- After connecting vertex x_{new} to the tree, check alternative connections to all potential parents in ball radius r_n
- If a lower cost path found through different parent, add edge to new parent, delete edge to old parent
- If a lower cost path found through x_{new} as parent, add new/delete old
- r_n shrinks as n grows, according to

$$r_n = \gamma \left(\frac{\log n}{n} \right)^{\frac{1}{d}}$$

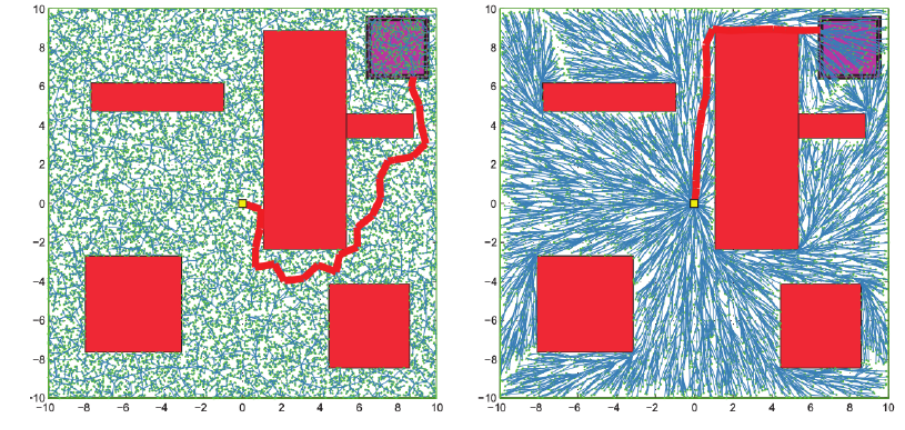


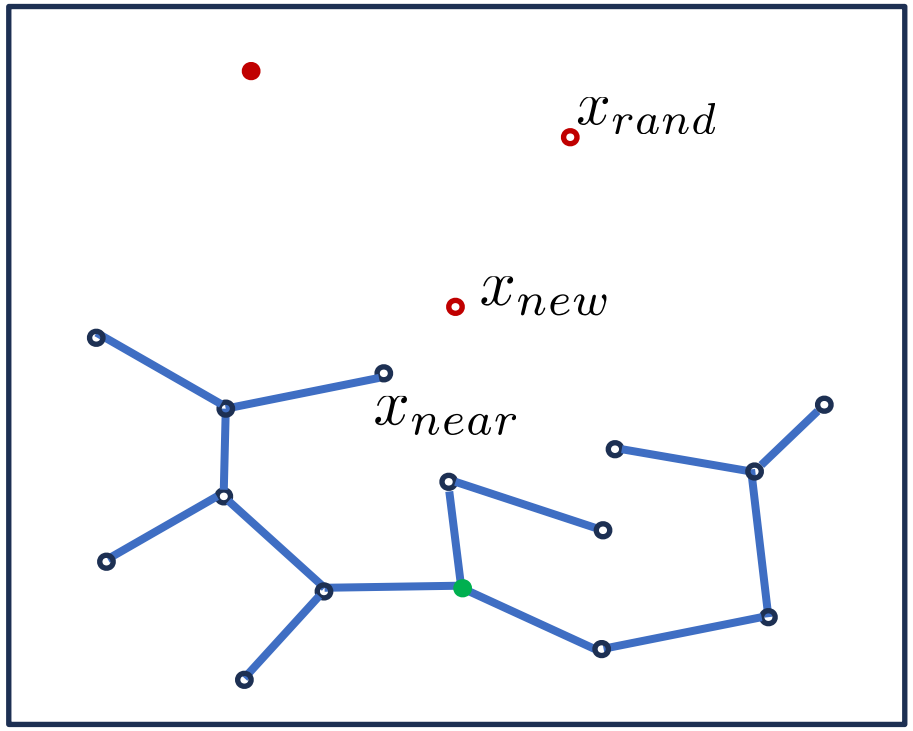


RRT* Re-wiring

- After connecting vertex x_{new} to the tree, check alternative connections to all potential parents in ball radius r_n
- If a lower cost path found through different parent, add edge to new parent, delete edge to old parent
- If a lower cost path found through x_{new} as parent, add new/delete old
- r_n shrinks as n grows, according to

$$r_n = \gamma \left(\frac{\log n}{n} \right)^{\frac{1}{d}}$$

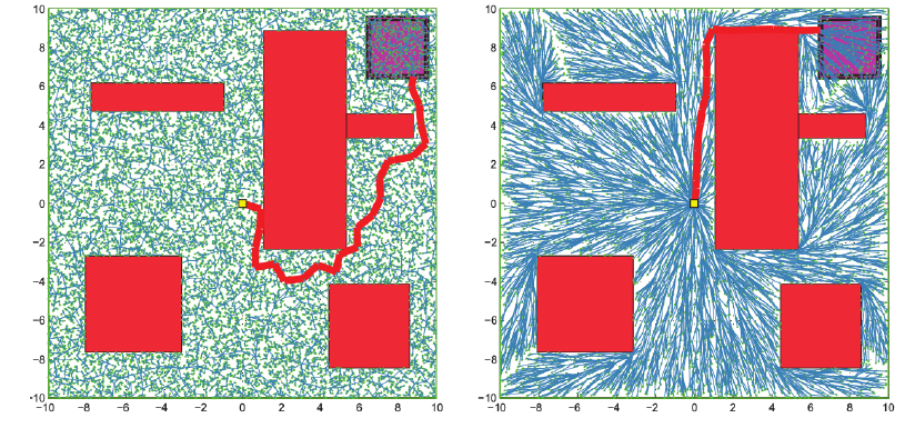


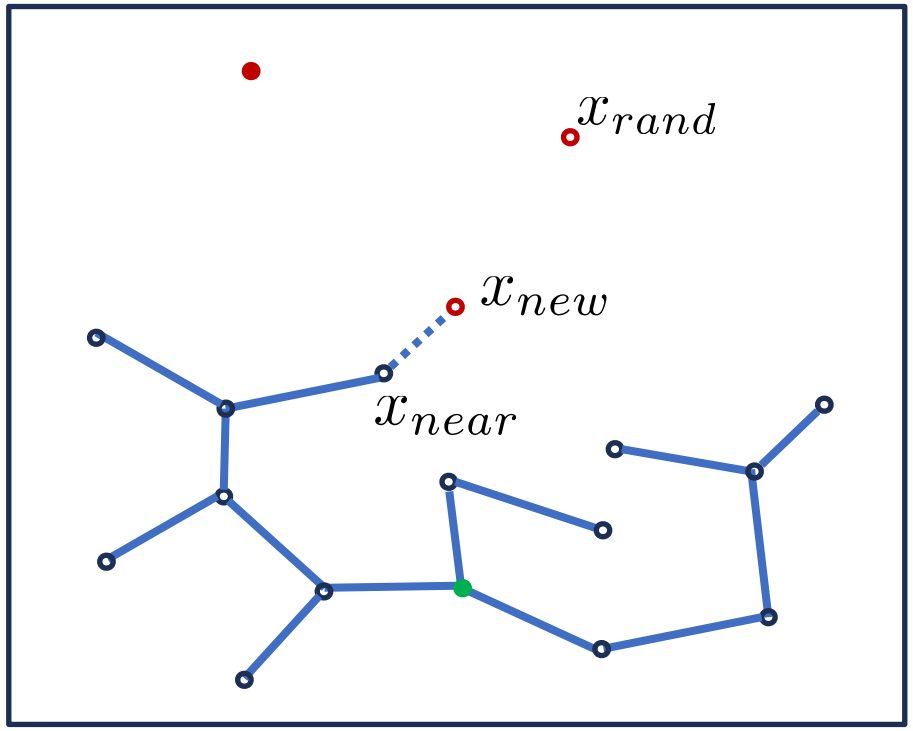


RRT* Re-wiring

- After connecting vertex x_{new} to the tree, check alternative connections to all potential parents in ball radius r_n
- If a lower cost path found through different parent, add edge to new parent, delete edge to old parent
- If a lower cost path found through x_{new} as parent, add new/delete old
- r_n shrinks as n grows, according to

$$r_n = \gamma \left(\frac{\log n}{n} \right)^{\frac{1}{d}}$$

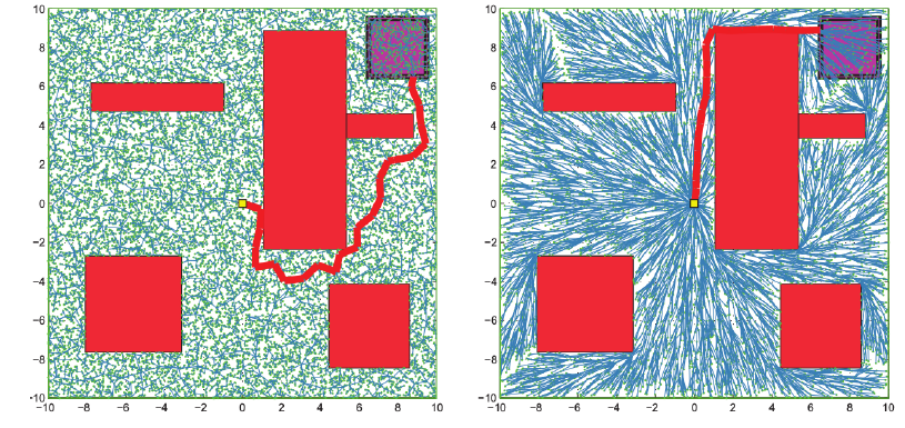


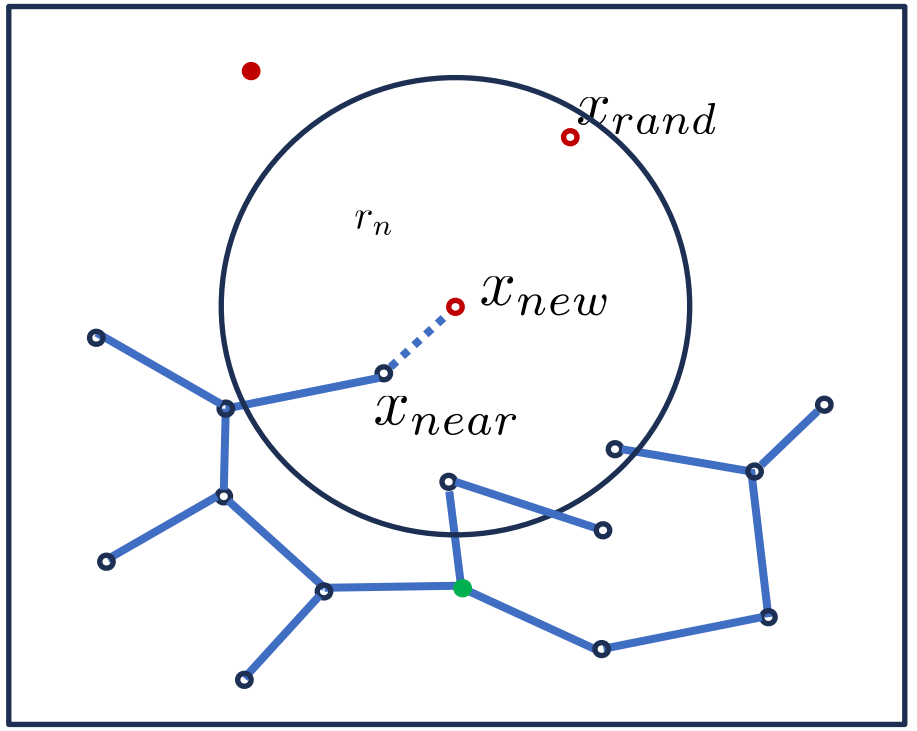


RRT* Re-wiring

- After connecting vertex x_{new} to the tree, check alternative connections to all potential parents in ball radius r_n
- If a lower cost path found through different parent, add edge to new parent, delete edge to old parent
- If a lower cost path found through x_{new} as parent, add new/delete old
- r_n shrinks as n grows, according to

$$r_n = \gamma \left(\frac{\log n}{n} \right)^{\frac{1}{d}}$$

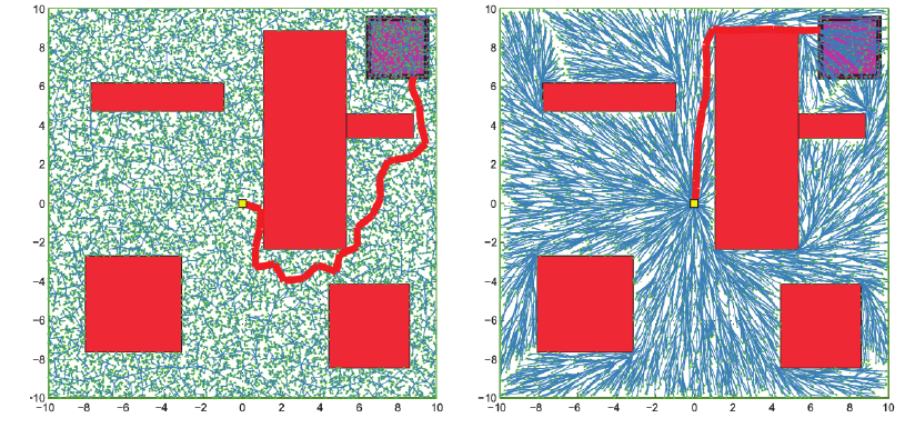


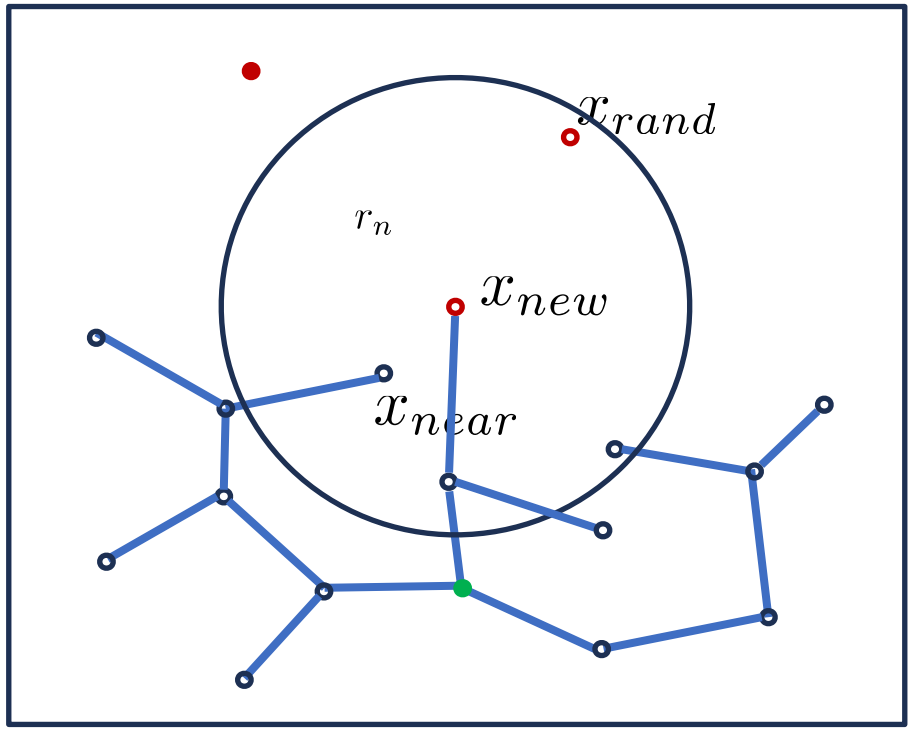


RRT* Re-wiring

- After connecting vertex x_{new} to the tree, check alternative connections to all potential parents in ball radius r_n
- If a lower cost path found through different parent, add edge to new parent, delete edge to old parent
- If a lower cost path found through x_{new} as parent, add new/delete old
- r_n shrinks as n grows, according to

$$r_n = \gamma \left(\frac{\log n}{n} \right)^{\frac{1}{d}}$$

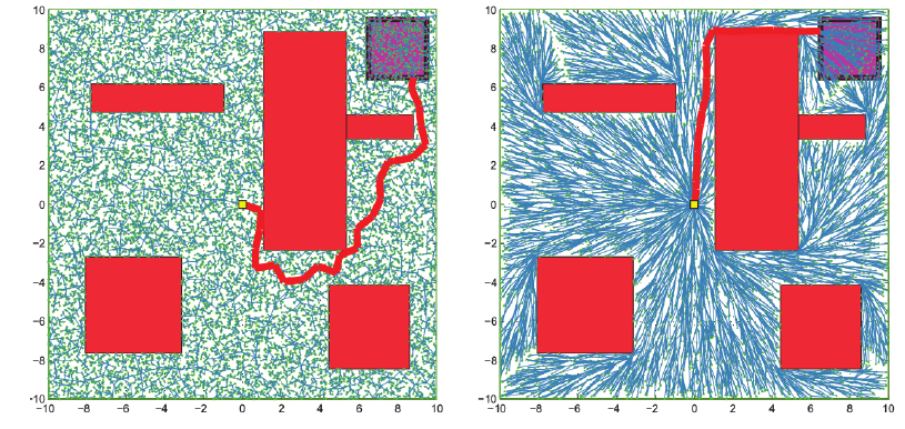


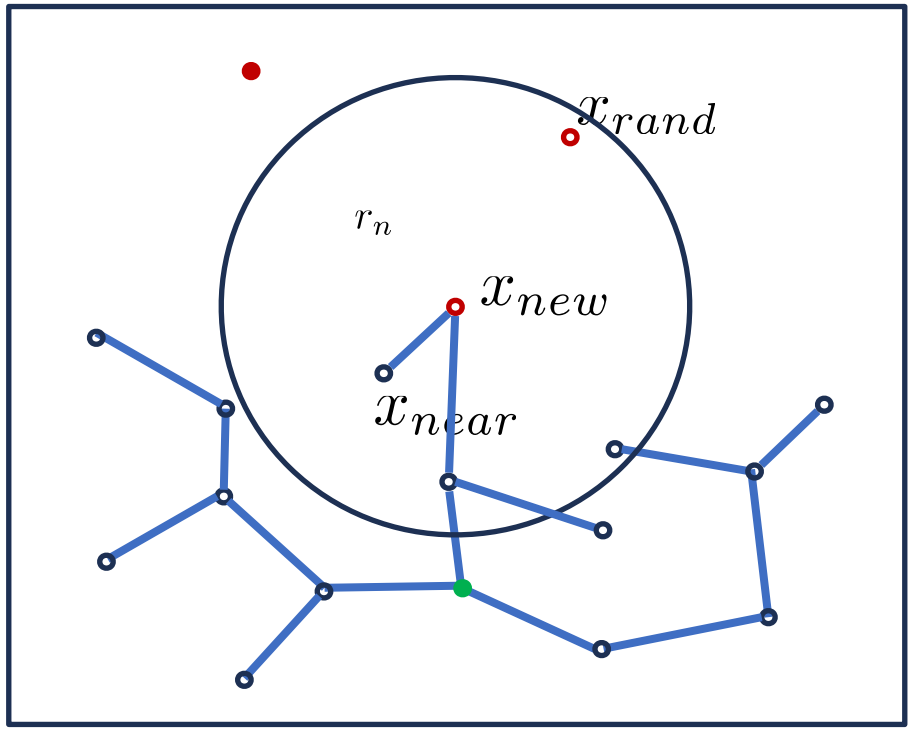


RRT* Re-wiring

- After connecting vertex x_{new} to the tree, check alternative connections to all potential parents in ball radius r_n
- If a lower cost path found through different parent, add edge to new parent, delete edge to old parent
- If a lower cost path found through x_{new} as parent, add new/delete old
- r_n shrinks as n grows, according to

$$r_n = \gamma \left(\frac{\log n}{n} \right)^{\frac{1}{d}}$$





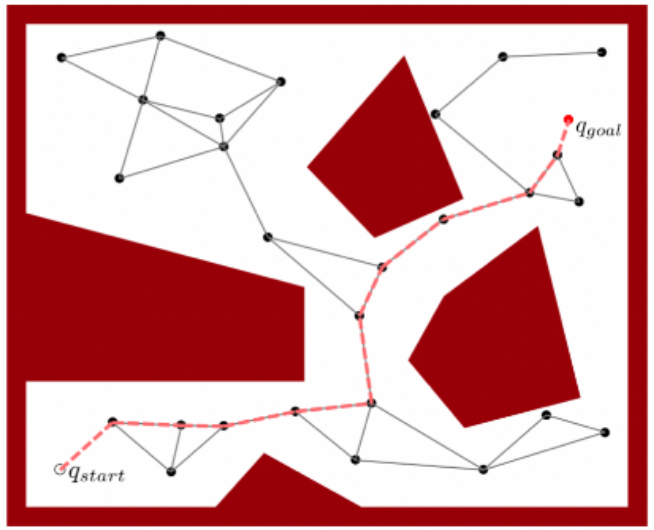
Observations

- PRM-like motion planning algorithms
  - For a given number of nodes n, they find “good” paths
  - ...however, require many costly collision checks

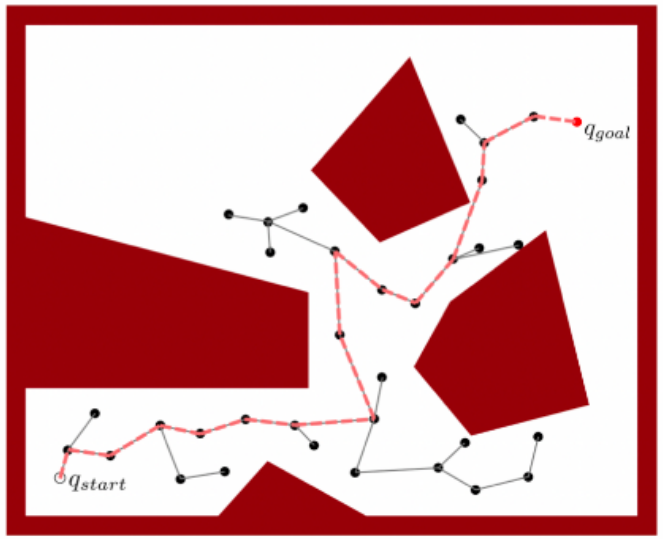
- RRT-like motion planning algorithms
  - Finds a feasible path quickly
  - ...however, the quality of that path is, in general, poor
  - “traps” itself by disallowing new better paths to emerge - RRT* performs local label correction as samples are added to help remedy this

Which of these two roadmaps has been created by RRT?

(A)



(B)

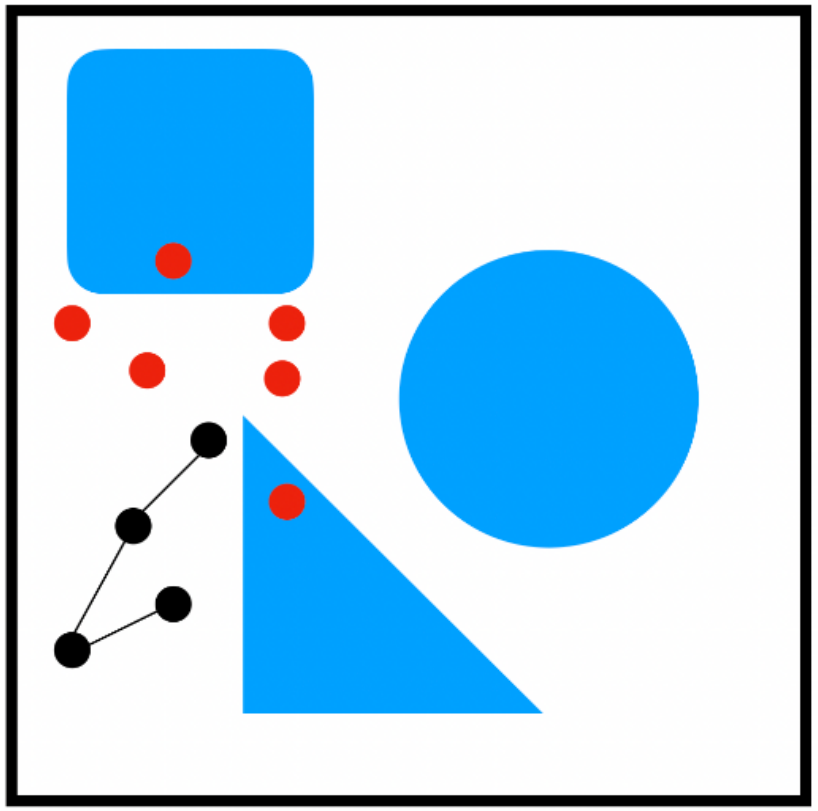


(A)

(B)

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

How many of the red sample points could lead to a new node in the roadmap constructed by RRT?



0

1

2

3

4

5

6

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Details and notes:

- Sampling-based planners transform the difficult global problem into a large set of local and easy problems
- A key ingredient is collision checking, which can be computationally time consuming. Speed ups avoid collision checking
- Local planning (edge validation) is typically performed by dense sampling of path and collision detection
- Another key ingredient is nearest-neighbor search (best done with k-d tree): given a query point find its nearest neighbor(s) within a set of points -- also well studied theoretically and practically

Sampling-based planning

Pros:

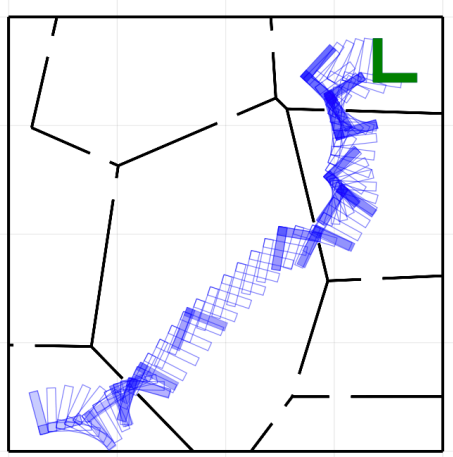
- Conceptually simple
- Relatively-easy to implement
- Flexible: one algorithm applies to a variety of robots and problems
- Beyond the geometric case: can cope with complex differential constraints, uncertainty, etc.

Cons:

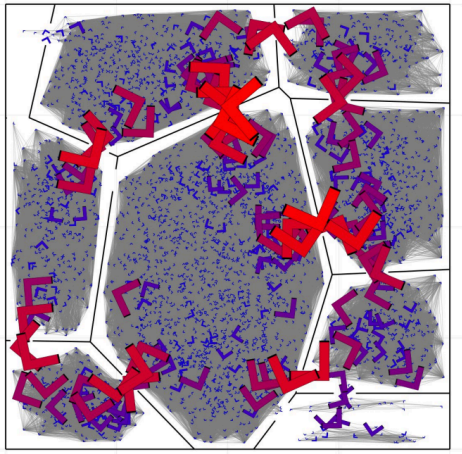
- Asymptotic
- Unclear how many samples should be generated to retrieve a solution
- Cannot determine whether a solution does not exist

Biased sampling for SBMP

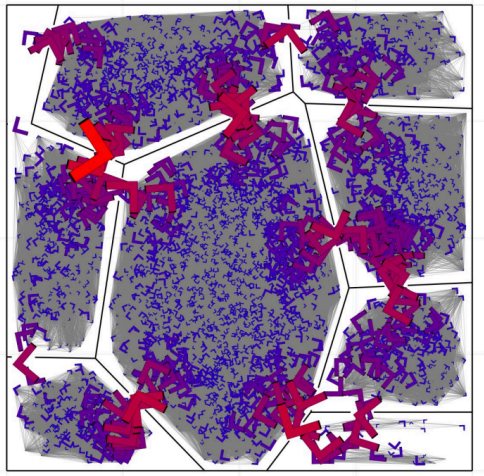
- Potential issue with uniform sampling: narrow corridors in C-space require many samples to identify/traverse
- Key idea: bias sampling towards suspected challenging regions of C-space
- Biased sampling distributions can be hand-constructed and/or adapt online (e.g., Hybrid Sampling PRM)
- Or learned from prior experience solving similar planning problems



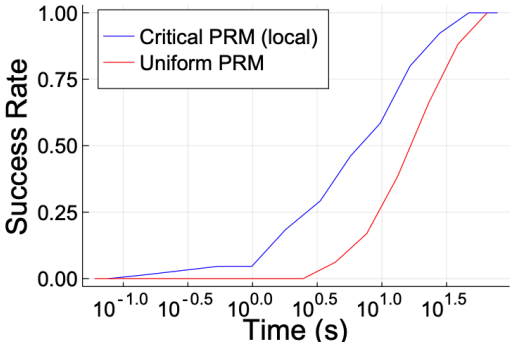
(a) SE(2) Planning



(b) Ground Truth Criticality



(c) Learned Criticality

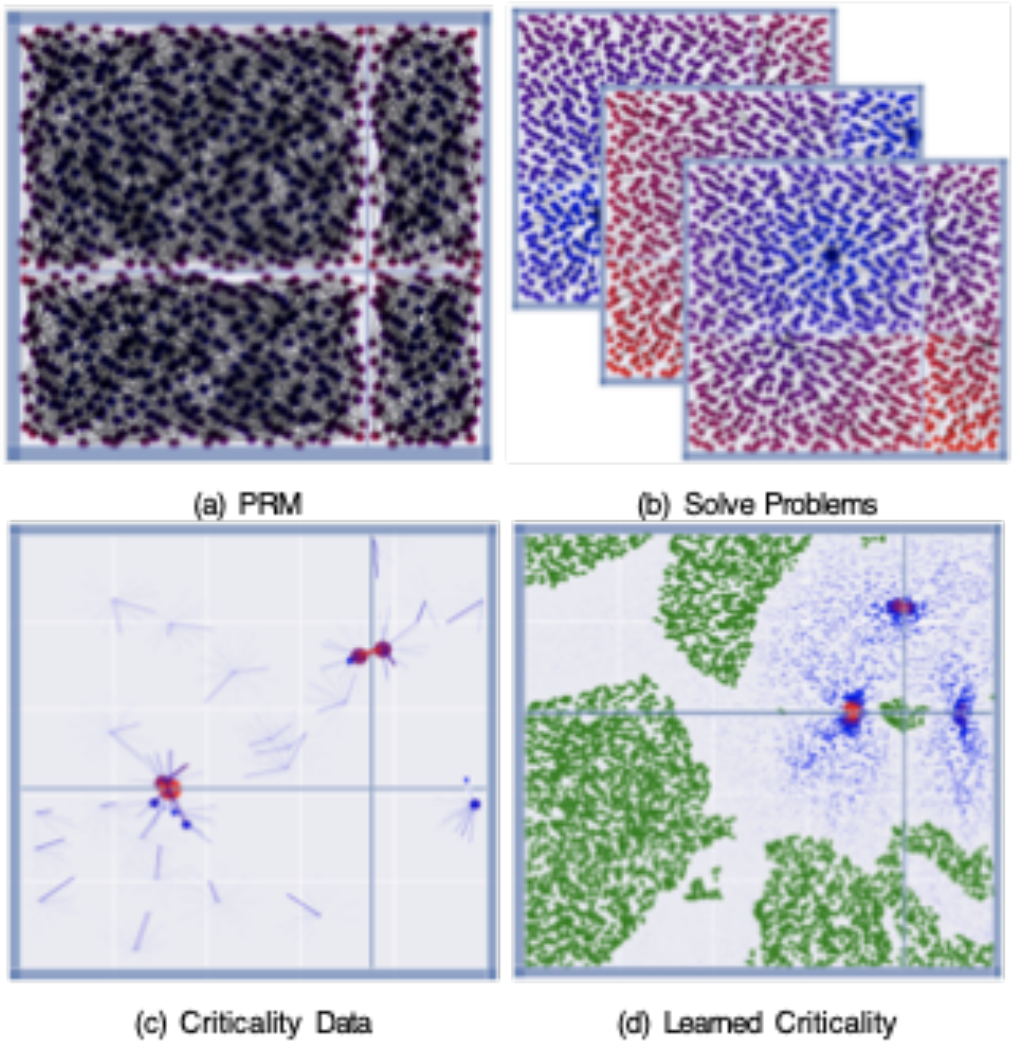


(d) Time (s) vs. Success

References:
Hsu et al. Hybrid PRM sampling with a cost-sensitive adaptive strategy. 2005.
Ichter et al. Learned Critical Probabilistic Roadmaps for Robotic Motion Planning. 2020

Learning criticality from local map features

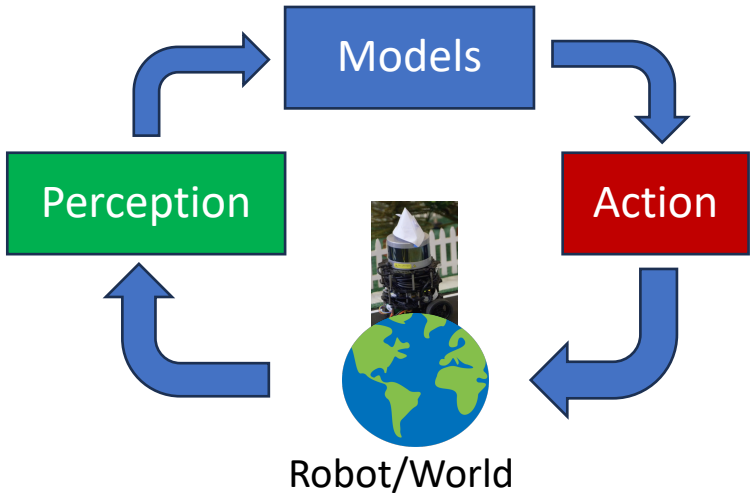
- On a collection of different maps, solve for optimal PRM paths between multiple start-goal pairs
- Score criticality of all nodes based on number of optimal paths passing through node
- Discount if node can be removed without collision
- Train CNN from local map patch centered at point to criticality of point



Ichter et al. Learned Critical Probabilistic Roadmaps for Robotic Motion Planning. 2020

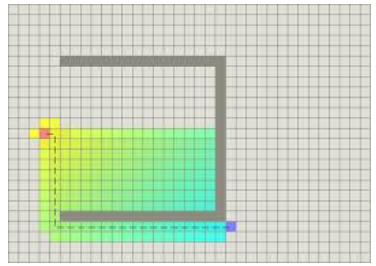
Action: Planning and Control Stack

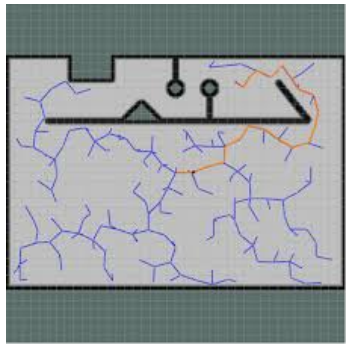
Use the models to move the robot



Path planning:

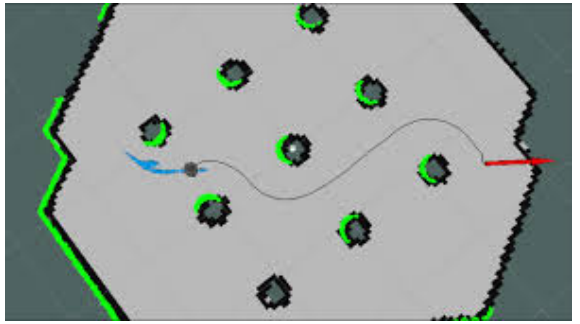
- A* planning on a graph
- RRT, RRT*





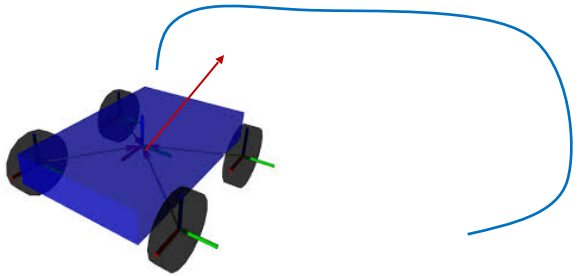
Trajectory optimization:

- Optimal control
- Numerical solvers



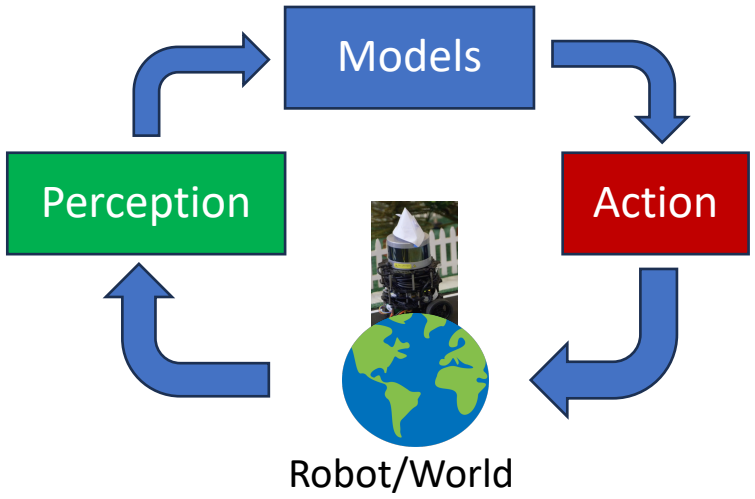
Trajectory following:

- Kinematic/dynamic models
- PID control
- LQR, gain scheduled LQR



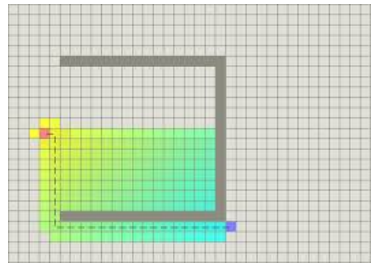
Action: Planning and Control Stack

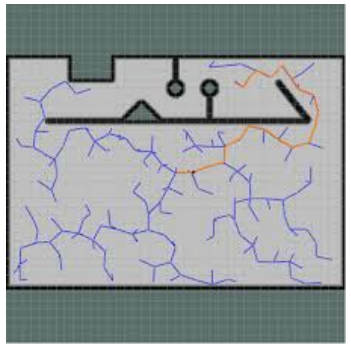
Use the models to move the robot



Path planning:

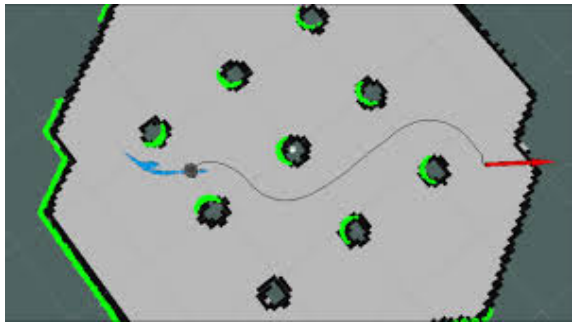
- A* planning on a graph
- RRT, RRT*





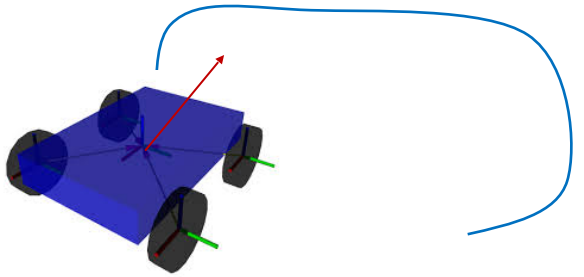
Trajectory optimization:

- Optimal control
- Numerical solvers



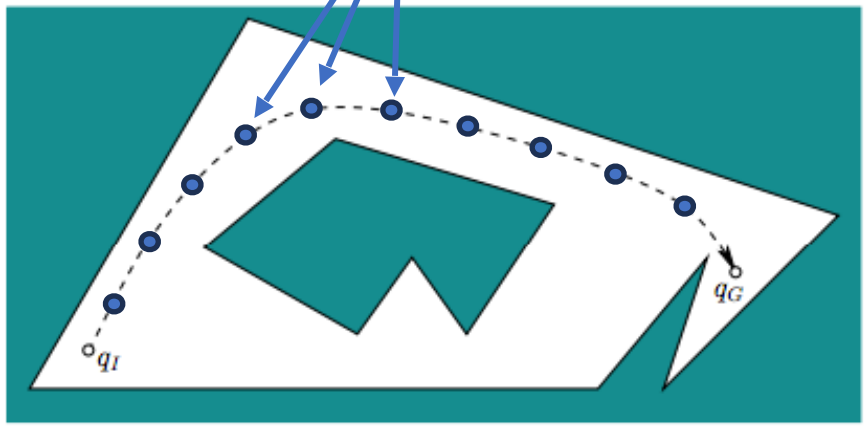
~~Trajectory following:~~

- Kinematic/dynamic models
- PID control
- LQR, gain scheduled LQR



Trajectory Optimization

Waypoint poses from path planner
(e.g. A*, RRT*)



Trajectory optimization problem:

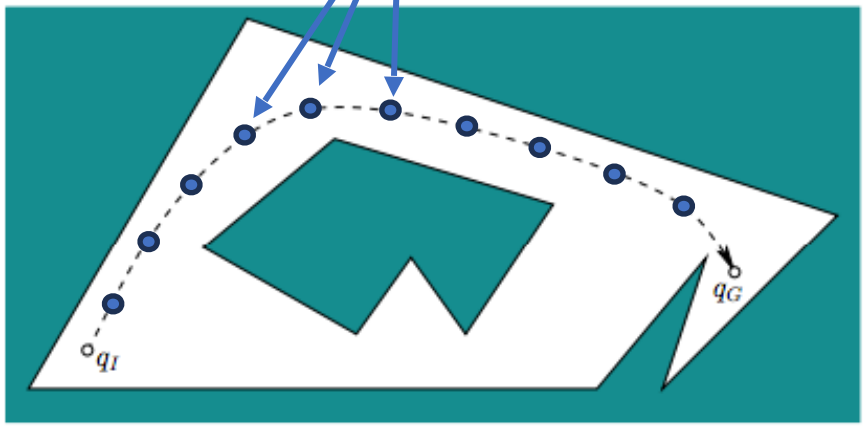
$$\min_{u_{0:T}} \sum_{t=0}^{T} c_t(x_t, u_t)$$

$$\text{s.t.} \quad x_{t+1} = f(x_t, u_t)$$

$$x_t \in \mathbb{X}_{\text{Free}}$$

Trajectory Optimization

Waypoint poses from path planner
(e.g. A*, RRT*)



Trajectory optimization problem:

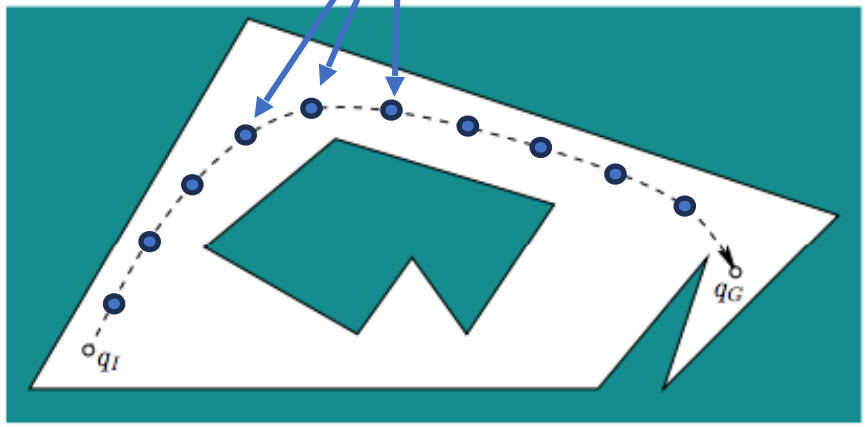
$$\min_{u_{0:T}} \sum_{t=0}^{T} \boxed{c_t(x_t, u_t)} \quad \text{Traj. cost}$$

$$\text{s.t.} \quad x_{t+1} = f(x_t, u_t)$$

$$x_t \in \mathbb{X}_{\text{Free}}$$

Trajectory Optimization

Waypoint poses from path planner
(e.g. A*, RRT*)



Trajectory optimization problem:

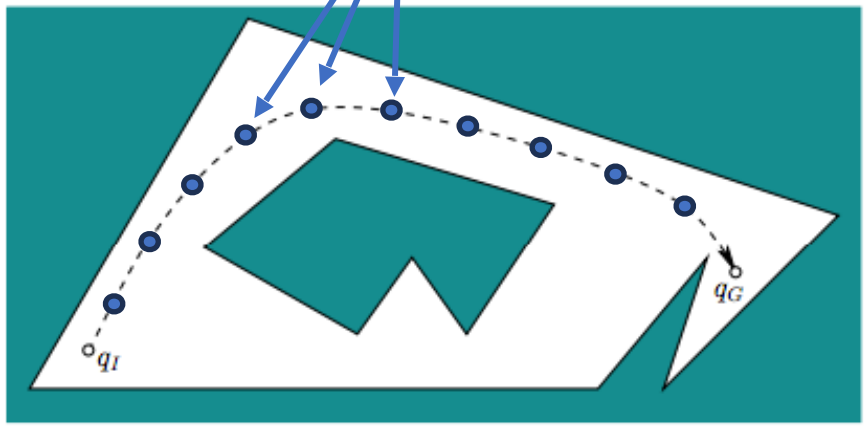
$$\min_{u_{0:T}} \sum_{t=0}^{T} \boxed{c_t(x_t, u_t)} \quad \text{Traj. cost}$$

$$\text{s.t.} \quad \boxed{x_{t+1} = f(x_t, u_t)} \quad \text{Dynamic constraints}$$

$$x_t \in \mathbb{X}_{\text{Free}}$$

Trajectory Optimization

Waypoint poses from path planner
(e.g. A*, RRT*)



Trajectory optimization problem:

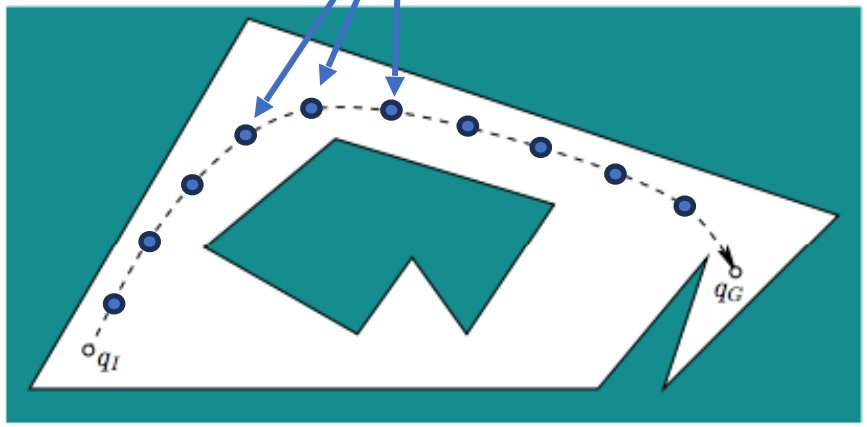
$$\min_{u_{0:T}} \sum_{t=0}^{T} c_t(x_t, u_t)$$ Traj. cost

$$\text{s.t.} \quad x_{t+1} = f(x_t, u_t)$$ Dynamic constraints

$$x_t \in \mathbb{X}_{\text{Free}}$$ Collision constraints

Trajectory Optimization

Waypoint poses from path planner
(e.g. A*, RRT*)



Trajectory optimization problem:

$$\min_{u_{0:T}} \sum_{t=0}^{T} c_t(x_t, u_t)$$ Traj. cost

$$\text{s.t.} \quad x_{t+1} = f(x_t, u_t)$$ Dynamic constraints

$$x_t \in \mathbb{X}_{\text{Free}}$$ Collision constraints

Goal: Find a dynamically feasible trajectory (close to the path) so the robot doesn't hit things!